

llama.cpp, Up Close

Structure, flow, and how to call it from C++, Rust, and Lua

Jonghwan Lee

2026

Contents

Preface	8
Who this book is for	8
How to read it	8
Conventions	9
What this book is not	9
What llama.cpp Is	9
The one-picture explanation	10
Why another inference runtime?	11
The vocabulary	11
A tiny end-to-end trip	12
What llama.cpp is not	13
The three things to take from this chapter	13
Exercises	13
Building llama.cpp	14
What the build produces	14
The three builds worth knowing	15
The flags you actually want to remember	16
Python helpers	17
Smoke-testing the build	17
Installing without building	17
Three things to take from this chapter	18
Exercises	18

First Run: llama-cli 18

The smallest useful command	19
The 10 flags that cover 80% of uses	19
The sequence, in one picture	19
Interactive chat mode	20
Reading the startup banner	21
A quick performance probe	21
Things that will go wrong the first time	22
Three things to take from this chapter	22
Exercises	23

The GGUF File Format 23

Layout in one picture	23
Why a new format at all?	25
Metadata, in practice	25
Tensor names: a small taxonomy	27
Producing a GGUF	28
When loading fails	29
Three things to take from this chapter	29
Exercises	30

The Code Map 30

One picture of the repo	30
Three layers, in dependency order	32
The shortest reading path	32
The ggml sub-tree, briefly	33

`common/`, `tools/`, `examples/` — pick the right one	33
Version discipline	34
Three things to take from this chapter	34
Exercises	34

The Forward Pass 35

The whole trip in one picture	35
Step 1: embedding	36
Step 2: attention	36
Step 3: feed-forward (SwiGLU)	37
Step 4: the final head	37
In code: the decode function	38
Prefill vs decode, again	38
Three things to take from this chapter	39
Exercises	39

Tensors and Backends 39

The layered picture	40
What a tensor is, in ggml	41
The graph	41
Backends	42
The scheduler	43
Op support	44
Memory and buffers	44
Three things to take from this chapter	44
Exercises	45

Context, Batch, and the KV Cache 45

The three-object picture	45
Creating a context	46
The KV cache, sized	47
Feeding a batch	47
Multi-sequence contexts	48
Continuous prompt reuse	48
Position indices matter	49
Three things to take from this chapter	49
Exercises	50

The Sampler Chain 50

The pipeline	50
The parameters that matter	51
Building a chain in C	52
Grammars (GBNF)	53
Debugging a weird output	54
Three things to take from this chapter	54
Exercises	55

Embedding llama.cpp in C++ 55

What we are building	55
CMake project	56
The full program	57
What the helpers do	59
Managing lifetimes	59
Threading model	60

Three things to take from this chapter	60
Exercises	60

Rust Bindings **61**

What the binding gives you	61
Cargo setup	62
The full program	62
Where Rust helps	64
Where it hurts	65
Async and server-shaped use	65
Alternative crates	65
Three things to take from this chapter	65
Exercises	66

Lua Bindings **66**

The shape of an FFI binding	66
Declaring the C API	67
A minimal wrapper module	69
The full program	71
Why LuaJIT	72
Gotchas	72
Three things to take from this chapter	73
Exercises	73

Appendix A: C API Cheat-Sheet **73**

Startup / teardown	73
Model	74

Context 74

Tokens 75

Batch 75

KV cache 75

Sampler chain 76

Freeing things 76

Appendix B: Glossary **77**

CHAPTER

Preface

This is a short book about **llama.cpp** with a narrow goal: show you how the code is shaped, how a single token flows through it, and how to drive it from **C++**, **Rust**, and **Lua**.

If you have already typed `./llama-cli -m model.gguf -p "hello"` and watched tokens stream back, you have seen the surface. The interesting part is what sits behind the CLI — a small, legible C/C++ core that you can link into your own programs in almost any language. That is what the second half of the book is about.

Who this book is for

- Engineers who can read C++ without panic and want to embed llama.cpp into their own app.
- Rustaceans and Lua hackers who want bindings that actually match how the C API is used.
- Curious readers who have heard "transformer", "quantization", and "KV cache" enough times to want to see them in an open source repository instead of a paper.

A GPU is nice but not required. Most of this book was drafted on a laptop, and every code example runs there.

How to read it

- **Chapters 1–3** — what llama.cpp is, how to build it, how to run the CLI. Skim if you've done this before.
- **Chapters 4–6** — the *structure* half. Repo layout, what a forward pass does, how ggml tensors and backends connect. Read in order; they build on each other.
- **Chapters 7–8** — the *flow* half. How context, batches, and the KV cache evolve token-to-token, and how the sampler chain turns logits into IDs.
- **Chapters 9–11** — one chapter per language. C++ is the native API. Rust uses the community `llama-cpp-2` crate. Lua is LuaJIT FFI against the shared library.

Each chapter has a diagram that earns its place on the page. Reproducible source lives in `scripts/figures/` ; if a picture looks wrong for your hardware, edit the script and regenerate.

Conventions

- Shell commands start with `$` .
- C++ code uses the names in `include/llama.h` . When the API changes (it does, regularly) I'll mark the version at the top of a snippet.
- Rust snippets target the `llama-cpp-2` crate as of early 2026.
- Lua snippets target LuaJIT 2.1+ (the FFI is much nicer than vanilla Lua's C API here).

What this book is not

A reference manual, a transformer textbook, or a benchmark survey. The `llama.cpp` wiki, Sebastian Raschka's LLM books, and the llama.cpp Discussions tab cover those better. This book is the "orient yourself in the codebase and call it from your language" piece that was missing in between.

— J. L., 2026

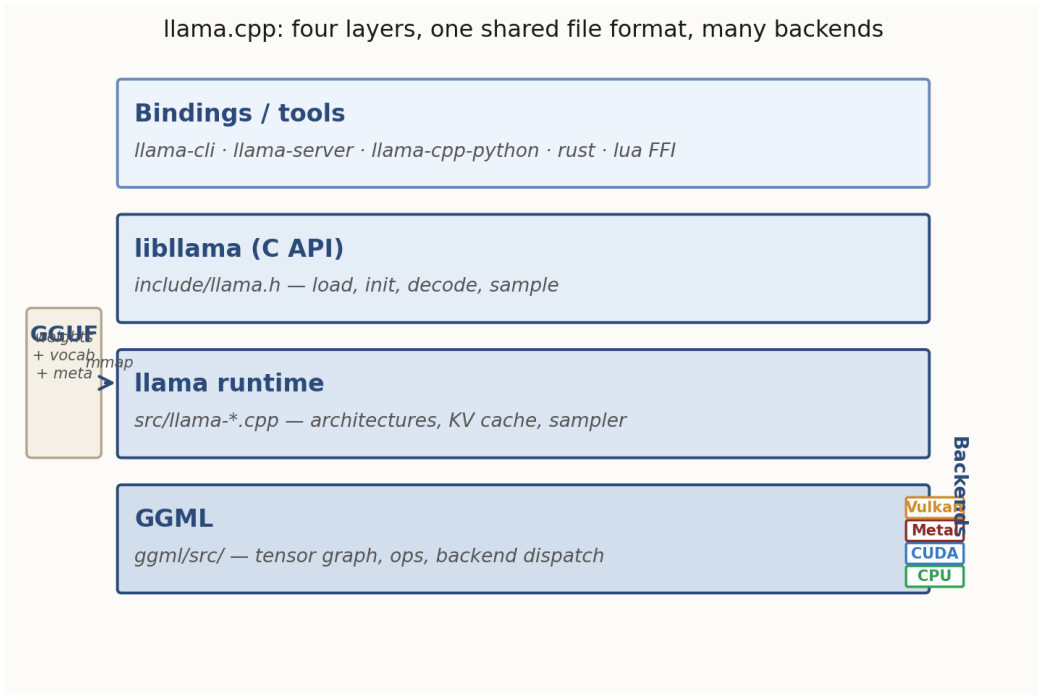
CHAPTER

What llama.cpp Is

Strip away the CLI, the server, and the dozens of frontends, and llama.cpp is a C/C++ library that does one thing: load a quantized transformer from disk and stream tokens out of it. The file on disk is called a GGUF. The library that does the arithmetic is called GGML. Everything else — the `llama-cli` binary, the HTTP server, the Python bindings, the desktop apps — is a thin wrapper around those two ideas.

This chapter draws the whole picture in one diagram, names the pieces, and tells you which three sentences you must remember before the rest of the book makes sense.

The one-picture explanation



llama.cpp at a glance: a GGUF file is memory-mapped, loaded into GGML tensors, and stepped through a transformer graph by a thin llama runtime; user code talks to it via a C API, a CLI, or an HTTP server.

Four layers stack on top of each other:

- **GGUF**, a self-describing file format that carries weights, metadata, and the tokenizer. One file per model, append-only layout, designed to be `mmap`'d straight into memory.
- **GGML**, a tensor library written in C. It defines a computation graph, a handful of backends (CPU, CUDA, Metal, Vulkan, SYCL, HIP/ROCm, CANN), and the quantized kernels that make the whole thing fast on weak hardware.
- **llama runtime** (`libllama`), a C/C++ layer on top of GGML that knows how to interpret a GGUF file as a specific transformer architecture and how to drive sampling, caching, and batching.
- **Tools and bindings**: `llama-cli` , `llama-server` , `llama-bench` , `llama-quantize` , plus third-party bindings like `llama-cpp-python` , `node-llama-cpp` , and language-native wrappers.

A user typically touches only the top layer. The reason the project rewards reading to the bottom is that a surprising number of real problems ("why is my token rate half what it should be?", "why does my fine-tune refuse to load?", "why does INT4 break at long context?") can only be diagnosed once you can see all four.

Why another inference runtime?

Local LLM inference is a crowded space: PyTorch with `transformers`, ONNX Runtime, TensorRT-LLM, vLLM, MLX, ExecuTorch, MLC-LLM, and a dozen research stacks. llama.cpp's specific sweet spot is:

1. **Zero-GPU viable.** On a laptop CPU with AVX2/NEON, a 7B model at Q4 quantization produces 10–30 tokens/sec. No Python, no CUDA, no cloud account.
2. **Small, legible C/C++ core.** The entire hot path is a few thousand lines. You can step through a forward pass in a debugger in an afternoon.
3. **Memory-mapped weights.** Start-up is milliseconds, and multiple processes share a single resident copy of the weights via the operating system's page cache.
4. **One file per model.** A GGUF carries the architecture, the hyperparameters, the tokenizer, and the quantized weights. You copy one file; the model runs.
5. **Backend portability.** The same GGUF runs on CPU, CUDA, Metal, Vulkan, SYCL, HIP, and CANN. The tensor graph does not care.

If you need batch-100 throughput with paged attention on an 8-GPU server, vLLM or TensorRT-LLM will beat llama.cpp. If you need one or two concurrent users on whatever hardware happens to be in front of you, llama.cpp is almost always the answer.

The vocabulary

A **model** is a GGUF file: one blob that contains the weights, the architecture ID (e.g. `llama`, `qwen2`, `gemma3`), the hyperparameters (layer count, hidden size, head count), and the tokenizer.

A **context** is the runtime state for one conversation: the KV cache, the pending batch of tokens, the sampler state. You can have several contexts open against the same model, and they will share memory-mapped weights but not the cache.

A **batch** is a set of tokens submitted together for evaluation. llama.cpp can run a single user's prompt as one batch, or interleave several users' tokens into a larger one (this is what

the server does, under the name *continuous batching*).

A **backend** is a GGML target — the code that actually executes the matrix multiplies. You pick it at build time (`-DGGML_CUDA=ON` , `-DGGML_METAL=ON` , etc.). At runtime you decide how many layers to place on it with `--n-gpu-layers` .

A tiny end-to-end trip

Chapter 2 will show you how to build llama.cpp. Chapter 3 will show you the full CLI. For a taste, here is the shape of a session:

```
# Download a small GGUF (about 1.7 GB at Q4_K_M):
curl -LO https://huggingface.co/unsloth/Qwen2.5-1.5B-Instruct-GGUF/resolve/main/Qwen2.5-1.5B-Instruct-Q4_K_M.gguf

# Chat with it:
./llama-cli -m Qwen2.5-1.5B-Instruct-Q4_K_M.gguf \
    -p "Explain quicksort in two sentences." \
    -n 120 --temp 0.7
```

Inside the runtime, the following happens, in order:

1. The GGUF is opened and `mmap` 'd. No weights are read yet.
2. The tokenizer is reconstructed from the GGUF's vocabulary metadata.
3. The prompt is tokenized into `N_prompt` token IDs.
4. A forward pass runs over those IDs in a single batch, computing the KV cache entries for every layer.
5. A sampler turns the final logits into one next-token ID.
6. That ID is appended to the batch; a one-token forward pass produces the next one. Repeat until end-of-sequence or `-n` .

Everything in chapters 4 through 14 is a closer look at one of those steps.

Note

The broker-consumer split that some other systems have — "a server holds the state, a client asks for tokens" — does not exist in the base library. `libllama` is a linked-in dependency. `llama-server` puts HTTP on top, but for an embedded use case you can run the model inside your own process and never touch a socket.

What llama.cpp is not

- **Not a training framework.** You cannot backprop through a GGUF; the quantized weights are read-only by design. For fine-tuning, use PyTorch or Unsloth and convert the result back to GGUF.
- **Not a state-of-the-art batched server.** Its server is solid for tens of concurrent users; it is not trying to compete with vLLM at hundreds.
- **Not transformer-only.** The repo also contains Whisper-adjacent audio, Mamba/SSM code paths, and an increasingly generic "state-space / hybrid" execution path, though transformers remain the center of gravity.

The three things to take from this chapter

Tip

1. **A GGUF is one file.** Weights, metadata, tokenizer — everything needed to run the model, in a single `mmap`-friendly blob.
2. **GGML does the math, libllama knows the architecture.** The split is what lets new models land quickly.
3. **Quantization is the whole game.** Without it, 7B models would need 14 GB of RAM. With Q4, they need 4. Chapter 5 is why.

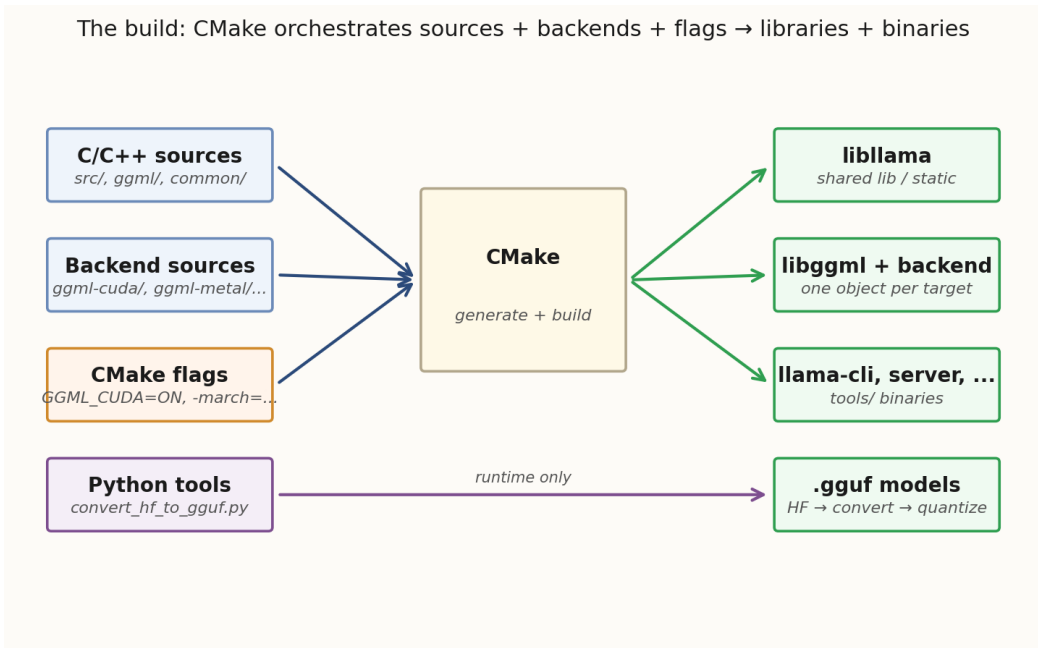
Exercises

1. Open a GGUF you already have with `xxd -l 128` (or just read the first 4 bytes with `head -c 4`). Which four ASCII characters do you see? Why would the file format need a magic number at all?
2. `llama.cpp` used to be called `ggml`-something. Skim `git log --reverse` in the repo and find the commit where the current name first appears. What problem does that rename solve?
3. Name three scenarios where you would reach for vLLM instead of llama.cpp and one where you would do the reverse.

Building llama.cpp

Every interesting problem in this book becomes easier if you can build the project yourself and run the binaries out of the build directory. Prebuilt wheels and Homebrew formulae exist, but they lag mainline by weeks and they hide the build options you actually want to understand. This chapter shows the three builds I reach for most often — a plain CPU build, a CUDA build on Linux, and a Metal build on macOS — and then names the flags that matter when you outgrow the defaults.

What the build produces



The build flow: sources compile to libllama and a handful of CLI tools; the selected GGML backend gets linked in from a separate object; the resulting binaries consume GGUFs at runtime.

From a clean clone, CMake produces:

- `libllama.{a, so, dylib}` — the runtime library every tool links.

- `libggml.{a,so,dllib}` plus one backend object per enabled target (`ggml-cpu` , `ggml-cuda` , `ggml-metal` , `ggml-vulkan` , ...).
- The CLI binaries: `llama-cli` , `llama-server` , `llama-quantize` , `llama-bench` , `llama-embedding` , `llama-perplexity` , `llama-tokenize` , `llama-imatrix` , `llama-speculative` , and more.
- Python helpers, most importantly `convert_hf_to_gguf.py` (pure Python; no compile step).

The point of the build is to pick which backend object gets baked in and which ISA extensions the CPU path is allowed to use. Everything else — model, prompt, quantization type — is decided at runtime.

The three builds worth knowing

Plain CPU build

Works on every machine, including continuous-integration runners:

```
git clone https://github.com/ggml-org/llama.cpp
cd llama.cpp
cmake -B build -DGGML_NATIVE=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
```

`GGML_NATIVE=ON` compiles with `-march=native` , which auto-detects AVX2/AVX512/NEON on the current host. If you plan to ship binaries to a different CPU, turn this off and set the ISA flags yourself (`-DGGML_AVX2=ON` `-DGGML_AVX512=OFF`) so the binary runs on the lowest common denominator.

CUDA build on Linux

Requires a recent CUDA toolkit (12.x at time of writing) and a host GCC that matches it. The CMake script auto-detects a sane `CMAKE_CUDA_ARCHITECTURES` , but you can pin it:

```
cmake -B build -DGGML_CUDA=ON \
  -DCMAKE_CUDA_ARCHITECTURES="75;80;86;89;90" \
  -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
```

The resulting `llama-cli` still runs on CPU if no GPU is visible; `-ngl` controls how many transformer layers to offload at runtime (see chapter 10). Building with CUDA does not forbid running on CPU.

Metal build on macOS

No flags needed — on Apple silicon the default build enables Metal:

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
```

The Metal backend uses unified memory, so there is no "host → device" copy; the weights are `mmap`'d and the GPU reads the same pages the CPU does. On an M-series Mac with 16 GB unified memory, you can run a 7B Q4 model without any tuning.

The flags you actually want to remember

<code>-DCMAKE_BUILD_TYPE=Release</code>	<code># always. Debug is ~5x slower.</code>
<code>-DGGML_NATIVE=ON</code>	<code># -march=native; local builds only.</code>
<code>-DGGML_CUDA=ON</code>	<code># NVIDIA GPUs.</code>
<code>-DGGML_METAL=ON</code>	<code># Apple silicon (default on macOS).</code>
<code>-DGGML_VULKAN=ON</code>	<code># AMD/Intel GPUs, Android, Raspberry Pi.</code>
<code>-DGGML_HIP=ON</code>	<code># AMD ROCm.</code>
<code>-DGGML_SYCL=ON</code>	<code># Intel GPUs via oneAPI.</code>
<code>-DGGML_BLAS=ON</code>	<code># Optional BLAS for CPU prompt eval.</code>
<code>-DGGML_OPENMP=ON</code>	<code># OpenMP threading on the CPU backend.</code>
<code>-DGGML_CCACHE=ON</code>	<code># Cache object files across rebuilds.</code>
<code>-DLLAMA_CURL=ON</code>	<code># Enables -hf model downloads at runtime.</code>

`-DBUILD_SHARED_LIBS=ON` gives you `.so` / `.dylib` instead of a static library — useful when you are linking llama.cpp into another program and want plugin-style backend loading.

Warning

`-DGGML_NATIVE=ON` and Docker do not mix. The container may be built on a Zen 4 host and run on a Haswell laptop, where AVX-512 will fault. Either pin your ISA flags or turn `GGML_NATIVE` off in container builds.

Python helpers

Some tools are Python, not C++:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

`requirements.txt` pulls `numpy`, `sentencepiece`, `protobuf`, `transformers`, and `gguf` (the Python GGUF library, independently installable). You need this environment to run `convert_hf_to_gguf.py` and the `gguf-py/` tools — the conversion from Hugging Face safetensors to GGUF is Python-only.

Smoke-testing the build

A three-line sanity check:

```
./build/bin/llama-cli --version
./build/bin/llama-bench -m some-model.gguf -p 64 -n 64
./build/bin/llama-server -m some-model.gguf --port 8080 &
curl -s http://localhost:8080/v1/models | jq .
```

`llama-bench` is the quickest way to confirm that the backend you compiled in is actually being used. It prints a table of prompt throughput (`pp`, prefill) and generation throughput (`tg`, decode) per build configuration. If the CUDA build shows the same numbers as the CPU build, your `-ngl` is probably zero (default).

Installing without building

For a quick tour you can skip this chapter:

- **macOS:** `brew install llama.cpp`
- **Linux (x86_64):** prebuilt tarballs on the GitHub Releases page.
- **Containers:** `ghcr.io/ggml-org/llama.cpp:full` ships CUDA and CPU variants.
- **Python wrapper:** `pip install llama-cpp-python` compiles llama.cpp against your local CUDA/Metal. Chapter 9 revisits this.

These are fine for exploration, but the rest of the book assumes `./build/bin/llama-cli` exists — all flags and paths are named accordingly.

Three things to take from this chapter

Tip

1. **CPU build first.** Always works, always useful for sanity checks.
2. **Backend is compile-time, offload is runtime.** `-DGGML_CUDA=ON` bakes CUDA in; `-ngl 30` decides what runs on the GPU.
3. `GGML_NATIVE` **is for local, not distributed, binaries.** CI, Docker, and release tarballs need explicit ISA flags.

Exercises

1. Build twice: once with `GGML_NATIVE=ON`, once with a fixed set of ISA flags. Diff the resulting `llama-bench` numbers on a long prompt. How much does `-march=native` actually buy you?
2. On Linux, watch `ls -l build/bin/*.so` after `cmake --build .` Which files change when you flip `GGML_CUDA` on and off?
3. Run `cmake -LA build` on a finished build directory. Which variables look like things you would want to pin in a release pipeline, and which are safe to leave as defaults?

CHAPTER

First Run: llama-cli

With a build in hand, the shortest path to a generating model is `llama-cli`. It takes a GGUF, a prompt, and a handful of sampling flags, and streams tokens back to the terminal. Everything you will do later — servers, pipelines, embeddings, benchmarks — is the same sequence of internal steps with a different wrapper on top. This chapter is the one where you get that sequence under your fingers.

The smallest useful command

Grab a small model and hit it with a one-shot prompt:

```
# 1.5B Qwen, Q4_K_M, about 1 GB on disk:
curl -LO https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF/resolve/main/qwen2.5-1.5b-instruct-q4_k_m.gguf

./build/bin/llama-cli \
  -m qwen2.5-1.5b-instruct-q4_k_m.gguf \
  -p "Write a haiku about compilers." \
  -n 64 --temp 0.7 --top-p 0.9 --seed 42
```

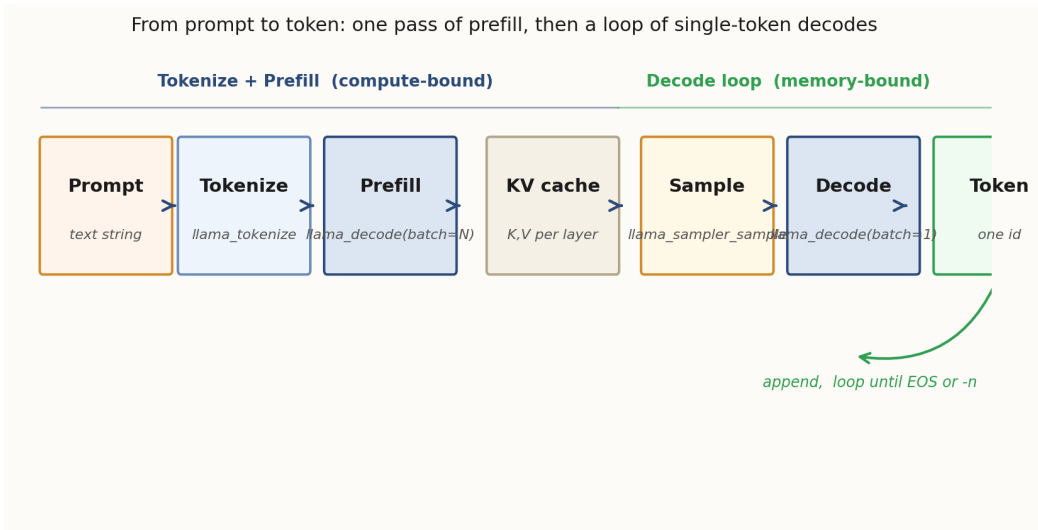
You should see the prompt echoed, a block of statistics, a short poem, and then a final line that looks like `llama_print_timings: eval time = ...`. The full sequence is prompt → tokenize → prefill → decode → print → stats.

The 10 flags that cover 80% of uses

-m PATH	GGUF model file. Always required.
-p "TEXT"	Initial prompt (single-shot mode).
-n N	Max new tokens to generate (-1 = until EOS).
-c N	Context size; must fit model's trained ctx.
-t N	CPU threads for prompt eval. Default: #cores.
-ngl N	Offload N layers to GPU (needs a GPU build).
--temp F	Temperature (0 = greedy, >1 = more random).
--top-p F	Nucleus sampling cut-off. Default 0.9.
--top-k N	Keep top-N candidates before sampling. 40 is typical.
--seed N	RNG seed; -1 = random each run.

The `--help` text lists dozens more, including grammar, bias, penalties, and caching flags. Chapters 6 and 8 will return to sampling and caching in depth.

The sequence, in one picture



The journey of a prompt: text is tokenized, the prompt batch is prefilled in parallel, each generated token runs a one-step decode using the KV cache from earlier tokens, and the sampler chooses the next ID. The loop repeats until EOS or the -n limit.

Two distinct phases matter to performance:

- **Prefill / prompt evaluation.** All prompt tokens go through the transformer in a single batch. This is where a GPU backend helps most: prefill is compute-bound.
- **Decode / token generation.** Each output token is a batch of size 1 against the growing KV cache. Decode is memory-bandwidth bound. A faster memory channel beats a bigger GPU here.

`llama-cli` prints the two throughput numbers separately at the end:

```

llama_perf_context_print: prompt eval time = 185 ms / 24 tokens
                        (7.70 ms/tok, 129.73 t/s)
llama_perf_context_print:      eval time = 1,842 ms / 96 tokens
                        (19.19 ms/tok, 52.12 t/s)

```

The `prompt eval` line is prefill; `eval` is decode. Ratios between the two change dramatically with backend, quantization, and context length — watching them drift is how you catch regressions.

Interactive chat mode

Pass `-cnv` to enter chat mode:

```
./build/bin/llama-cli -m qwen2.5-1.5b-instruct-q4_k_m.gguf -cnv
```

The CLI reads the chat template embedded in the GGUF's metadata (if any) and applies it to turn your input into the format the model was instruction-tuned on. Type a message, hit Enter, and the model responds. Ctrl-C ends the session. `--chat-template` lets you override the model's default template; chapter 7 covers the topic in detail.

Reading the startup banner

The first 30 lines of output are diagnostic gold. A representative excerpt:

```
llama_model_load: loading model from qwen2.5-1.5b-instruct-q4_k_m.gguf
llama_model_loader: Dumping metadata keys/values.
llama_model_loader: - kv  0: general.architecture      str  = qwen2
llama_model_loader: - kv  1: general.name              str  = Qwen2.5-1.5B-
llama_model_loader: - kv  2: qwen2.block_count         u32  = 28
llama_model_loader: - kv  3: qwen2.context_length      u32  = 32768
...
print_info: file format = GGUF V3 (latest)
print_info: file type   = Q4_K - Medium
print_info: file size   = 940.31 MiB (4.95 BPW)
```

Three numbers are worth circling:

- `block_count` : number of transformer layers. Relevant when you set `-ngl` (you usually want either 0 or `block_count + 1`, and `block_count + 1` includes the output layer).
- `context_length` : the maximum context the model was trained or fine-tuned on. The `-c` flag must be $\leq$ this value.
- `BPW` : bits per weight. 4.95 is typical for Q4_K_M; Q8_0 is 8.5; F16 is 16.

A quick performance probe

Benchmarks belong in `llama-bench`, but for a rough check during experimentation, re-run with a fixed seed and compare timings:

```
./llama-cli -m M.gguf -p "Write 100 words about bees." \
-n 100 --seed 42 2>&1 | tail -n 6
```

Do it once cold, once warm. The cold run pays the page-fault cost of loading the weights into the OS cache for the first time; the warm run is what you should quote. Anything larger than a 2× gap between cold and warm usually means you are swapping, not `mmap`ing.

Note

Output repeatability with the same seed and sampler settings is a useful sanity check. If two runs on the same build and model differ despite identical flags, either GPU reductions are non-deterministic (see `--no-gpu-offload-reductions` on some backends), or you changed the batch size, which can alter reduction order on the CPU too.

Things that will go wrong the first time

- `error: failed to load model`. The file is truncated (the HuggingFace download was interrupted) or not a GGUF. `head -c 4` the file — it must start with `GGUF`.
- **Garbage output.** Wrong chat template. Some base models produce Markov-chain noise without the right instruction wrapper.
- **Extremely slow prefill.** GPU build but `-ngl` is 0, so the prompt runs on the CPU.
- `invalid ctx_size`. You asked for `-c 32768` on a model whose GGUF declares `context_length=4096`. Chapter 8 covers RoPE scaling for going beyond that.

Three things to take from this chapter

Tip

1. **Prefill and decode are different workloads.** Benchmark them separately; tune them separately.
2. **The startup banner is documentation.** The first 30 lines answer most "why doesn't this work" questions.
3. **Seed, temperature, and top-p together determine reproducibility.** Pin all three when diffing outputs.

Exercises

1. Run the same prompt with `--temp 0`, `--temp 0.3`, `--temp 0.8`, and `--temp 1.5`. At which temperature does the model first start producing nonsense? Does that threshold depend on the prompt?
2. Measure prefill and decode throughput with `-ngl 0`, `-ngl 16`, and `-ngl 99` on a model with 28 layers. Plot both numbers. Which flag position maximizes decode t/s?
3. Use `--prompt-cache cache.bin` twice with the same prompt. How much faster is the second run? Inspect the cache file with `ls -lh`; does its size make sense?

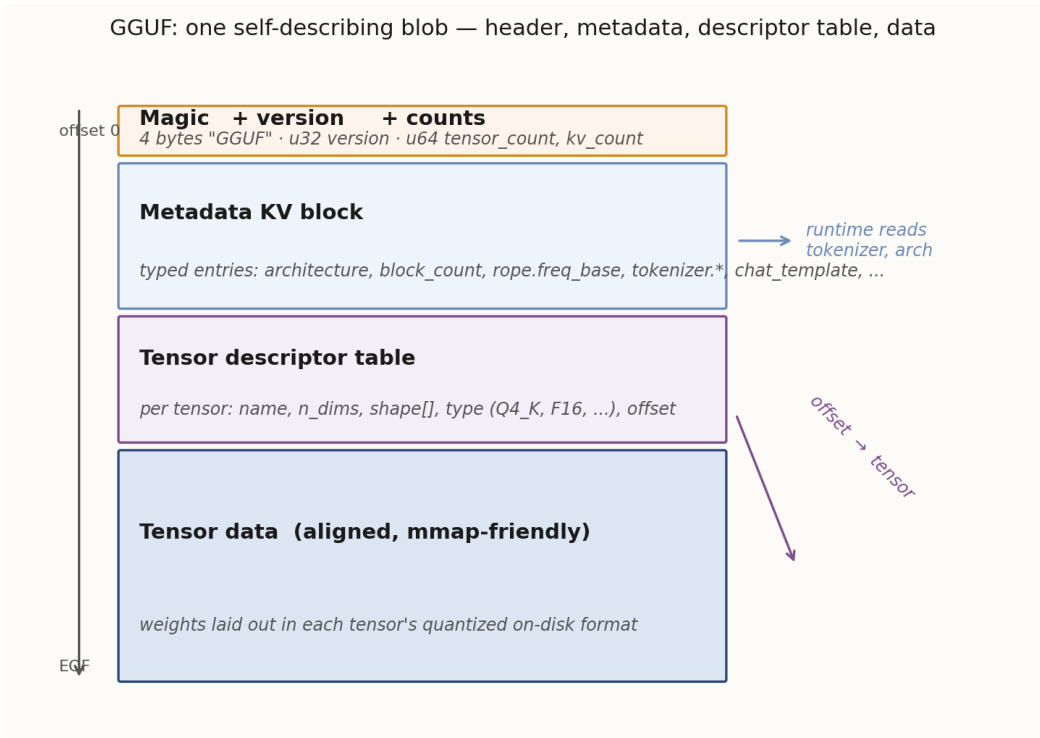
CHAPTER

The GGUF File Format

A GGUF file is a single self-describing blob on disk. Open it with `mmap`, read the header, and you know everything you need to reconstruct a transformer: the architecture, the hyperparameters, the tokenizer, and every weight tensor in its declared quantization. The rest of the runtime is book-keeping around that one data structure.

This chapter takes the file apart, explains why it is shaped the way it is, and shows how to inspect and produce one.

Layout in one picture



A GGUF file is a magic number, then a key-value metadata block, then a tensor descriptor table, then a large tensor data region aligned for mmap. The tokenizer, architecture, and hyperparameters all live in the metadata; the descriptor table tells the runtime where each weight starts and in what quantization.

Read top-to-bottom, a GGUF v3 file is:

1. **Magic** — the four ASCII bytes `GGUF`.
2. **Version** — a `uint32` (currently 3).
3. **Tensor count** — how many tensors follow.
4. **Metadata count** — how many key-value pairs follow.
5. **Metadata key-value pairs** — typed entries for every scalar, array, string, or nested type the writer wants to record.
6. **Tensor info table** — for each tensor: name, shape, type (quantization), and offset into the data region.
7. **Tensor data** — one large aligned region, with each tensor laid out in its own quantization's on-disk encoding.

The tensor data region begins at an offset aligned to `general.alignment` (default 32 bytes). This is what makes `mmap` possible: the OS maps the file into virtual memory, and the runtime treats every tensor as a pointer into that mapping.

Why a new format at all?

Before GGUF, the project used GGML and then GGJT, neither of which carried enough metadata. GGUF was introduced in August 2023 to fix three specific pains:

- **Self-contained models.** The tokenizer used to live in a separate file; mismatched pairs were an endless source of garbage-output bug reports.
- **Forward compatibility.** The metadata is typed and arbitrary — new architectures can carry their own hyperparameters without format changes.
- `mmap` **-friendly alignment.** Weights begin at a fixed alignment boundary, so reading a tensor is a pointer lookup, not a seek.

Most community GGUF files you encounter are v3. v2 still works; v1 is long gone.

Metadata, in practice

The metadata is a typed dictionary. Well-known keys include:

Key

Type

Meaning

`general.architecture`

string

`llama` , `qwen2` , `gemma3` , ...

`general.name`

string

Human-readable model name

`general.file_type`

u32

Overall quantization (Q4_K_M...)

```
general.quantization_version
```

u32

Quant format version

```
<arch>.block_count
```

u32

Number of transformer layers

```
<arch>.context_length
```

u32

Training context window

```
<arch>.embedding_length
```

u32

Hidden size

```
<arch>.attention.head_count
```

u32

Attention heads

```
<arch>.attention.head_count_kv
```

u32

KV heads (GQA models have fewer)

```
<arch>.rope.freq_base
```

f32

RoPE base (theta)

```
tokenizer.ggml.model
```

string

```
gpt2 , llama , qwen2 , ...
```

```
tokenizer.ggml.tokens
```

array<str>

Vocabulary (ordered)

```
tokenizer.ggml.merges
```

array<str>

BPE merge rules

```
tokenizer.chat_template
```

string

Jinja2 template

The runtime only requires a few of these (architecture, layer count, hidden size, head counts); the rest are consumed by tokenizer or by specific features (RoPE scaling, chat formatting, GBNF bias).

Inspect them with the bundled tool:

```
./build/bin/llama-gguf model.gguf r
```

For a quick one-line summary:

```
./build/bin/llama-cli -m model.gguf -n 0 2>&1 | grep "print_info:"
```

Tensor names: a small taxonomy

Tensor names are strings, by convention following a layer pattern:

```
token_embd.weight
output_norm.weight
output.weight
blk.0.attn_norm.weight
blk.0.attn_q.weight
blk.0.attn_k.weight
blk.0.attn_v.weight
blk.0.attn_output.weight
blk.0.ffn_norm.weight
blk.0.ffn_gate.weight
blk.0.ffn_up.weight
```

```
blk.0.ffn_down.weight
blk.1.attn_norm.weight
...
```

`blk.N.*` enumerates the N-th transformer block. Inside each block you see the attention projections (`attn_q` , `attn_k` , `attn_v` , `attn_output`) and the feed-forward network (`ffn_gate` , `ffn_up` , `ffn_down` — the SwiGLU trio used by most modern models). Norms (`attn_norm` , `ffn_norm`) round out each block. The list is the shape of the network; a change here means an architecture variant.

Producing a GGUF

The canonical path from a Hugging Face checkpoint is:

```
# 1. Get the weights (safetensors + tokenizer files):
git lfs install
git clone https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct

# 2. Convert to a full-precision GGUF:
python3 convert_hf_to_gguf.py Qwen2.5-1.5B-Instruct \
    --outfile qwen2.5-1.5b.f16.gguf \
    --outtype f16

# 3. Quantize to your target size:
./build/bin/llama-quantize qwen2.5-1.5b.f16.gguf \
    qwen2.5-1.5b-q4_k_m.gguf Q4_K_M
```

Step 2 is a Python reader/writer: it decodes the safetensors, copies the tokenizer, and emits a GGUF with weights still in F16 (or F32 or BF16). Step 3 is a C++ tool that reads the full-precision GGUF and writes a quantized one. You almost always want both: the full-precision copy is small enough to keep on disk and big enough to re-quantize to any target later without reloading the original Hugging Face weights.

```
# The conversion can also be driven from Python, useful in scripts:
from pathlib import Path
import subprocess

src = Path("Qwen2.5-1.5B-Instruct")
f16 = Path("qwen2.5-1.5b.f16.gguf")
q4  = Path("qwen2.5-1.5b-q4_k_m.gguf")
```

```
subprocess.run([
    "python3", "convert_hf_to_gguf.py", str(src),
    "--outfile", str(f16), "--outtype", "f16",
], check=True)

subprocess.run([
    "./build/bin/llama-quantize", str(f16), str(q4), "Q4_K_M",
], check=True)
```

When loading fails

Three common failure modes, in order of frequency:

- **"unknown model architecture"**. `general.architecture` names a string the runtime has no loader for. Usually the upstream model is newer than your build; update llama.cpp and retry.
- **"vocabulary size mismatch"**. Tokenizer and output-projection dimensions disagree. Almost always because the GGUF was produced against a different Hugging Face snapshot than the tokenizer files alongside it.
- **"tensor X has wrong shape"**. An architecture variant the loader does not yet recognize (e.g. new GQA head layout). Often fixed in a recent commit.

Warning

`general.file_type` is an overall tag, but individual tensors can be stored in different quantizations. `Q4_K_M`, for instance, stores some matrices in `Q4_K` and some in `Q6_K`. Never infer a tensor's quantization from the file-level tag — always check its per-tensor type in the descriptor table.

Three things to take from this chapter

Tip

1. **GGUF is one file.** Weights, tokenizer, and chat template all travel together.
2. **Metadata is typed and extensible.** New architectures can ship without a format version bump.
3. **Tensor types are per-tensor.** The file-level name is a label, not a guarantee.

Exercises

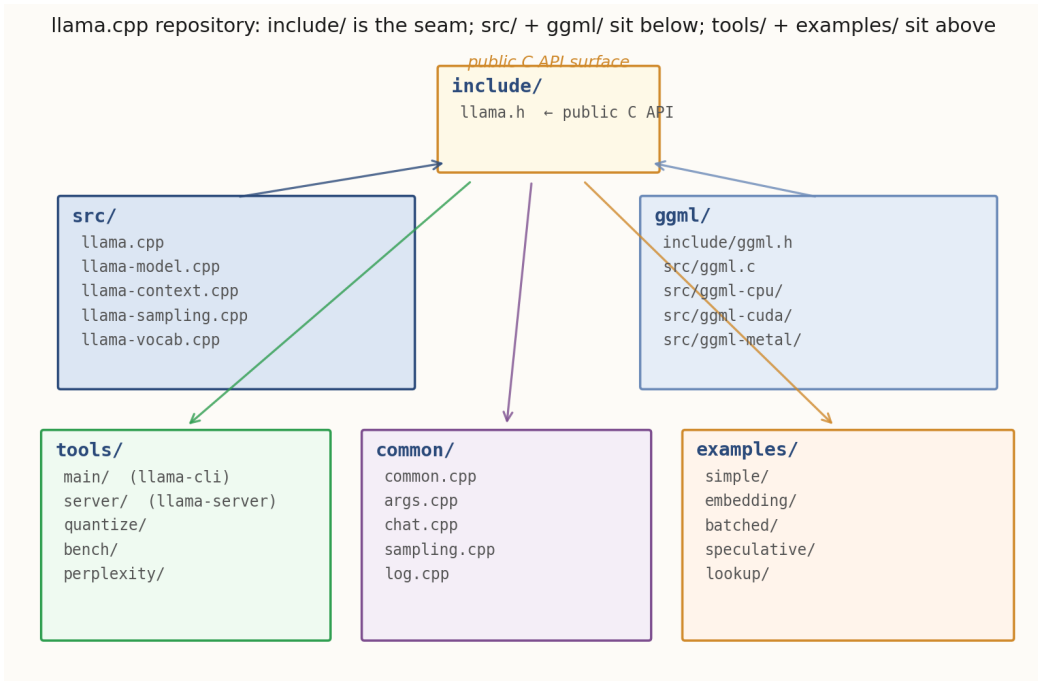
1. Run `llama-gguf model.gguf r` and count the tensors. Match the count to `block_count × (per-block tensors) + embedding + output + norms`. Do the numbers add up?
2. Dump the chat template with `jq` from the metadata JSON you get from the server's `/props` endpoint (chapter 9). Which model in your collection has the most verbose template?
3. Take a 4B-parameter model's F16 GGUF. Predict its file size from first principles before you run `ls -lh`, then check. Where does the gap come from — metadata, alignment padding, embedding tensors?

CHAPTER

The Code Map

Before you call llama.cpp from another language, or trace a bug into its internals, it helps to have a mental map of the repository. The whole thing is about 250k lines of code, but the *interesting* part — the one you need to understand to follow a forward pass end-to-end — is maybe 15k lines across six directories. This chapter names them.

One picture of the repo



Top-level layout: ggml/ is the tensor library and backends; src/ is the llama runtime (model loaders, KV cache, sampler); include/llama.h is the public C API; common/ has the shared helpers used by tools/; tools/ houses llama-cli, llama-server, llama-quantize and friends.

From the top of the clone, the directories that matter:

```

llama.cpp/
■■■ include/
■   ■■■ llama.h           ← the public C API (~500 lines)
■■■ src/
■   ■■■ llama.cpp         ← model load, eval, KV cache
■   ■■■ llama-model.cpp   ← architecture → graph construction
■   ■■■ llama-sampling.cpp ← sampler chain implementations
■   ■■■ llama-vocab.cpp   ← tokenizer runtime
■   ■■■ llama-context.cpp ← the runtime state object
■■■ ggml/
■   ■■■ include/ggml.h    ← the tensor API
■   ■■■ src/
■       ■■■ ggml.c        ← tensor graph, core ops
■       ■■■ ggml-cpu/     ← CPU kernels + threading
■       ■■■ ggml-cuda/    ← NVIDIA backend
■       ■■■ ggml-metal/   ← Apple backend
  
```

```

■      ■■■■ ggml-vulkan/    ← Vulkan backend
■■■■ common/
■      ■■■■ common.cpp      ← arg parsing, prompt helpers
■■■■ tools/
■      ■■■■ main/           ← llama-cli
■      ■■■■ server/         ← llama-server
■      ■■■■ quantize/       ← llama-quantize
■      ■■■■ ...
■■■■ convert_hf_to_gguf.py  ← HF → GGUF converter
■■■■ examples/             ← small standalone programs

```

Everything else — `docs/` , `scripts/` , `tests/` , `ci/` — is support.

Three layers, in dependency order

GGML is at the bottom. It does not know what a transformer is; it only knows how to build a computation graph of tensor ops (`matmul` , `add` , `softmax` , `rms_norm` , `rope` , `flash_attn`) and dispatch that graph to a backend (CPU, CUDA, Metal, Vulkan). If you delete the `src/` directory (everything starting with `llama-`), GGML still compiles and still runs; you just have no LLM on top.

The llama runtime (`src/llama-*.cpp`) is the transformer-aware layer. It knows that `blk.0.attn_q.weight` is a query projection, that `qwen2` needs RoPE with a specific `freq_base` , that GQA models have fewer KV heads than Q heads. For each supported architecture, it builds a specific tensor graph out of GGML primitives and feeds it a prompt.

The public C API (`include/llama.h`) is a thin set of opaque handles and functions. If you open `llama.h` in one window and `src/llama.cpp` in another, most entry points are one-to-one: you find `llama_model_load_from_file` , `llama_init_from_model` , `llama_decode` , `llama_sampler_sample` . That header is what every other language's binding is wrapping.

The shortest reading path

To read the codebase in one sitting, follow the call chain from the CLI:

1. `tools/main/main.cpp::main()` — argument parsing, banner.
2. `llama_model_load_from_file` (`src/llama.cpp`) — GGUF open, mmap, metadata parse, tensor table.

3. `llama_init_from_model` — allocate the context, allocate the KV cache, set up the compute buffers.
4. `llama_tokenize` — prompt string → token IDs.
5. `llama_decode` — one batch of tokens forward; returns logits.
6. `llama_sampler_sample` — logits → one token ID.
7. Loop back to step 5 with a batch of size 1 until EOS.

Six function calls, four files. That is the whole story; the rest of the book is a closer look at what happens inside each.

The ggml sub-tree, briefly

`ggml/src/` has one C file per backend. A backend is a collection of implementations for a fixed set of ops, plus scheduling code that knows which ops to run where (e.g. keep `norm` on CPU, push `matmul` to GPU). The file naming is consistent:

<code>ggml-cpu/ggml-cpu.c</code>	← CPU backend dispatch
<code>ggml-cpu/ggml-cpu-quants.c</code>	← quantized matmul for CPU
<code>ggml-cuda/*.cu</code>	← CUDA kernels (per-op)
<code>ggml-metal/ggml-metal.m</code>	← Apple Metal backend
<code>ggml-vulkan/ggml-vulkan.cpp</code>	← Vulkan shader dispatch

Backends are chosen at compile time (CMake flags from chapter 2) and loaded at runtime. Since ggml 0.1.x the backend layer is pluggable: the same build can register CPU, CUDA, and Metal, and the graph scheduler decides per-op where to run.

`common/` , `tools/` , `examples/` — **pick the right one**

- `common/common.cpp` is the helper library every CLI tool links. It owns argument parsing (`gpt_params`), chat-template selection, and sampler construction from flags. If you want to copy a CLI's behavior into your own program, this is what you copy.
- `tools/` holds the production binaries. They aim for breadth: every flag, every format, every edge case. Read them for completeness, not for simplicity.
- `examples/` has the small, self-contained programs. Start there when you want to learn; `examples/simple/simple.cpp` is about 60 lines and runs end-to-end.

Tip

When a feature you want is not exposed in a binding, check whether it lives in `common/` rather than the public API. `common/` is not part of the stable C interface, so many bindings re-implement its helpers in the host language instead of FFI-ing through it.

Version discipline

The public C API is versioned by `LLAMA_API_VERSION` in `include/llama.h`. It changes more often than you might expect — monthly, sometimes weekly — because the project is still adding features. When you pin a llama.cpp commit in a downstream project, pin the exact SHA; `HEAD` is not an API.

The Rust and Lua chapters will show how to handle that reality in practice; the short answer is "wrap the C API in a thin layer of your own, and update that layer on every bump."

Three things to take from this chapter

Tip

1. **ggml, llama, and the API are three layers.** GGML does math; `src/llama-*.cpp` knows transformer architectures; `llama.h` is the binding surface.
2. **The six-call path is the whole thing.** Load, init, tokenize, decode, sample, loop.
3. **Read** `examples/simple/simple.cpp` **first.** Sixty lines, no frills, end-to-end.

Exercises

1. `git grep -l "llama_decode" | wc -l`. How many files call it? Pick three and sketch how each wraps the call.
2. Open `include/llama.h` and count the opaque `struct` types (`llama_model`, `llama_context`, ...). Those are the only handles any binding needs to expose.

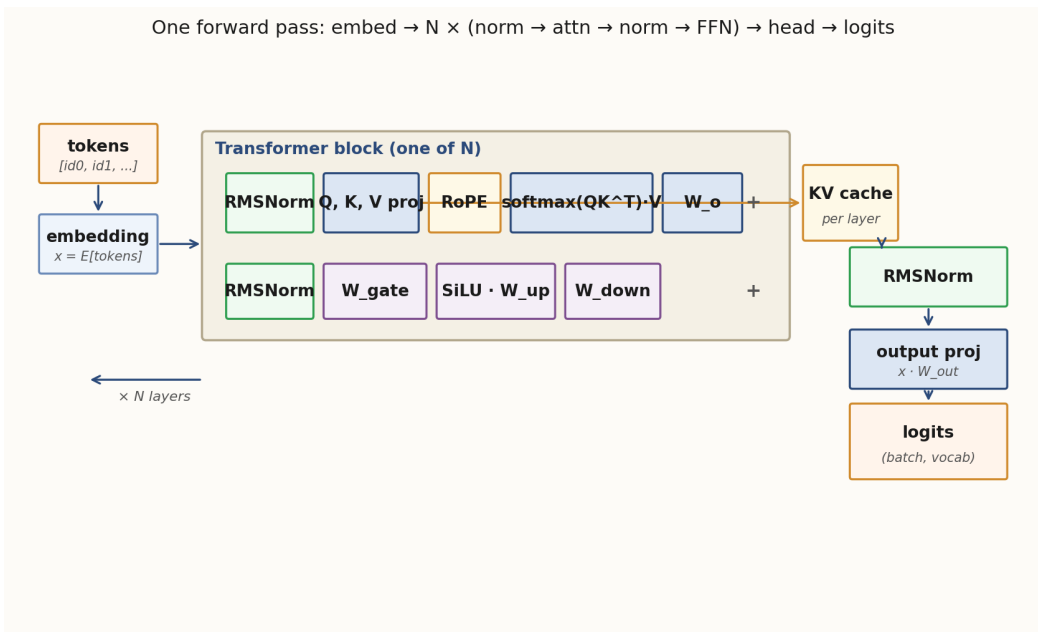
3. Compare `tools/main/main.cpp` with `examples/simple/simple.cpp` side-by-side. Which lines in `main` are optional for a minimum viable decoder?

CHAPTER

The Forward Pass

"Run a token through the model" is a sentence that hides a lot. This chapter walks the full path — from a `llama_decode` call down to the matrix multiplies that produce logits — using a concrete Qwen-shaped model as the example. By the end you will be able to read any architecture's graph-construction function in `src/llama-model.cpp` and predict what it does.

The whole trip in one picture



One forward pass: token IDs enter, the embedding table turns them into vectors; each transformer block does norm $\rightarrow$ attention (Q/K/V projections, rotary embeddings, softmax, output projection) $\rightarrow$ norm $\rightarrow$ feed-forward (gate/up/down SwiGLU); a final norm and output projection produce logits over the vocabulary.

From the outside, a forward pass is this simple:

```
token IDs  → [embedding]
           → [block_0] → [block_1] → ... → [block_N-1]
           → [final RMSNorm]
           → [output projection]
           → logits (shape: batch × vocab)
```

Every *block* inside the stack has the same two-halves shape:

```
x  → RMSNorm → Attention(Q, K, V) → +x  (residual)
   → RMSNorm → SwiGLU_FFN         → +x  (residual)
```

The architecture-specific file in `src/` (e.g. the `llama` or `qwen2` case in `llama-model.cpp::llm_build`) reads exactly like that: build the embedding, loop over blocks, add the head.

Step 1: embedding

Tokens go in as integers and come out as vectors. The embedding matrix has shape `(vocab, hidden)`; indexing it by a token ID returns the row. In code:

```
// Pseudocode – actual call is ggml_get_rows in the graph builder.
inp_embd = ggml_get_rows(ctx, model.tok_embd, inp_tokens);
// inp_embd has shape (n_tokens, hidden)
```

For a batch of `T` tokens and hidden size `H`, this is a single gather op. It is cheap; on a laptop it's under a millisecond even for long prompts.

Step 2: attention

Inside a block, attention is five linear algebra moves:

1. **Normalise:** `x_norm = RMSNorm(x)`.
2. **Project:** `Q = x_norm · W_q`, `K = x_norm · W_k`, `V = x_norm · W_v`.
3. **Rotate:** apply RoPE to `Q` and `K` (positional info).
4. **Attend:** `scores = softmax(Q · KT / √d_k) · V`.
5. **Project back:** `out = scores · W_o`.

`d_k` is the per-head dimension: `hidden / n_heads`. Most modern models use **grouped-query attention** (GQA), which means the model has fewer `K` and `V` heads than `Q` heads — typically 1 KV head per 4 or 8 Q heads. The reason is the KV cache (chapter 7): shrinking KV head count shrinks the cache proportionally.

Where the KV cache lives

The `K` and `V` projections for *all past tokens* are kept in the KV cache. On a new token, the model only computes `Q`, `K`, `V` for the new token, appends `K` and `V` to the cache, and attends against the whole cached history. That is why decode is a size-1 batch: no matter how long the context, you are only multiplying one new token's rows through the stack.

```
// Conceptual shape, one layer:
//   Kcache[layer] : (n_ctx, n_kv_heads, d_k)
//   Vcache[layer] : (n_ctx, n_kv_heads, d_k)
// At step t:
//   Kcache[layer][t] = Knew
//   Vcache[layer][t] = Vnew
//   attn = softmax(Qnew · Kcache^T) · Vcache
```

Step 3: feed-forward (SwiGLU)

Most modern LLMs use a gated feed-forward network with three matrices instead of two:

```
g = x · W_gate
u = x · W_up
h = silu(g) * u      ← elementwise
y = h · W_down
```

`silu(x) = x * sigmoid(x)`. The gate times up-projection trick ("Swi-GLU") is what `ffn_gate`, `ffn_up`, and `ffn_down` in the GGUF tensor table refer to. Two matmuls get replaced with three, but each is smaller; the net effect is better quality at the same parameter budget.

Step 4: the final head

After `N` blocks you have a tensor of shape `(T, hidden)`. Apply one more RMSNorm, then project to vocabulary size:

```
out = RMSNorm(x)
logits = out * W_output // shape (T, vocab)
```

Most models *tie* `W_output` to the transpose of the embedding matrix, saving one big chunk of parameters. You can see this in a GGUF by noting whether `output.weight` is present at all: if not, `token_embd.weight` is reused.

In code: the decode function

At the top level, `llama_decode` boils down to:

```
int llama_decode(llama_context *ctx, llama_batch batch) {
    // 1. Make sure KV cache has room for batch.n_tokens new slots.
    // 2. Build the compute graph for `batch` using model + kv_self.
    // 3. Allocate tensor buffers for the graph.
    // 4. Schedule the graph on the active backend (CPU/CUDA/Metal).
    // 5. Read back the logits into ctx->logits.
    return 0;
}
```

Each step has a source function with a parallel name in `src/llama.cpp`. For the simplest possible trace, instrument those five points with `fprintf(stderr, ...)` and run a one-prompt session; you'll see exactly where your time is going.

Prefill vs decode, again

Prefill runs the same graph above with `batch.n_tokens = N_prompt`; decode runs it with `batch.n_tokens = 1`. The graph structure is identical; the *shape* of the tensors flowing through it is not. At prefill, the attention softmax is an $N \times N$ matrix. At decode, it's $1 \times (N+t)$ for output token `t`. That single shape difference is why prefill is compute-bound and decode is memory-bound.

Note

A lot of "why is my model slow?" questions reduce to "you are running prefill in a loop instead of caching K and V." The KV cache is what makes per-token cost roughly constant with context length instead of linear.

Three things to take from this chapter

Tip

1. **A block is norm → attention → norm → FFN.** Memorise that and most architectures look the same.
2. **K and V are cached; Q is not.** The cache shrinks decode to one new row of arithmetic per token per layer.
3. **Prefill and decode use the same graph, different shapes.** Tune them as separate workloads.

Exercises

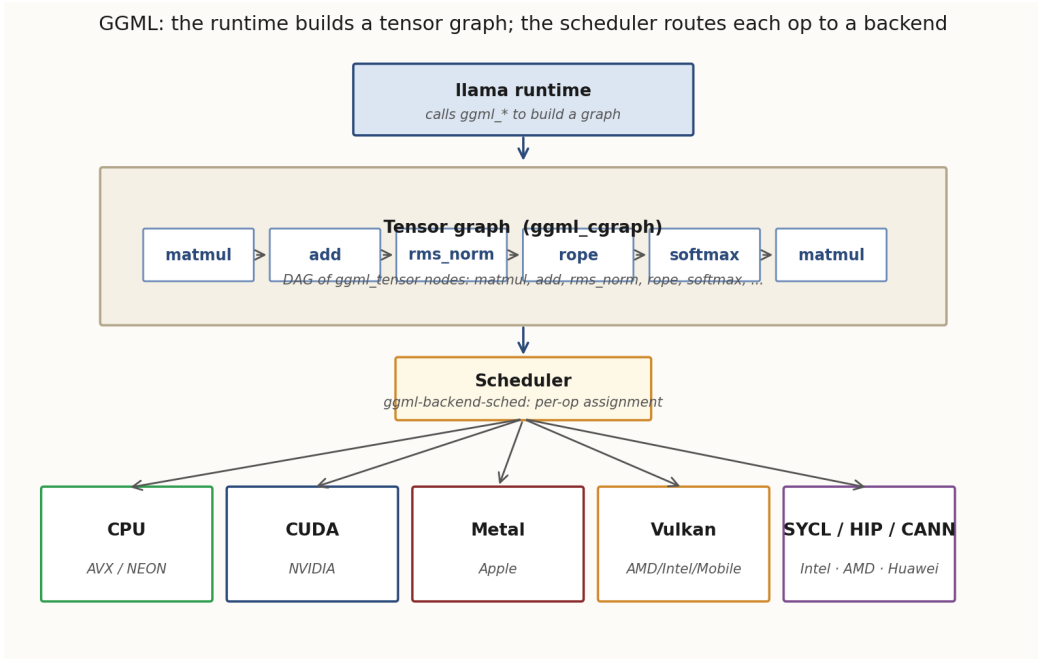
1. Open `src/llama-model.cpp` and find the case for `qwen2` (or `llama`). List the ggml ops it calls in order for one block. Match each to a step in this chapter.
2. For a model with `hidden=2048`, `n_heads=16`, `n_kv_heads=2`, compute the KV cache size in MB for 8192 tokens at F16. Where does the factor-of-8 reduction from GQA show up?
3. Instrument `llama_decode` with a timer at entry and exit. Plot latency vs `batch.n_tokens` for values 1, 8, 64, 512. The knee of that curve tells you your prompt's optimal chunk size.

CHAPTER

Tensors and Backends

If you want to understand how the same GGUF file runs on a CPU, an Apple GPU, and an NVIDIA card without any format change, you need a little of GGML's internal model. This chapter sketches the *tensor graph*, the *backend* abstraction, and the *scheduler* that decides which op runs where. It is the closest this book comes to low-level.

The layered picture



GGML sits under llama. The runtime builds a tensor graph; the scheduler assigns each op to a registered backend (CPU, CUDA, Metal, Vulkan, SYCL, HIP); the backend runs the op on its hardware and returns results to the shared context.

The stack from top to bottom:

- **llama runtime** decides what to compute — which transformer to build, which tokens to feed in.
- **GGML graph** is the data structure describing the computation: a DAG of `ggml_tensor` nodes linked by ops.
- **Scheduler** (`ggml-backend-sched`) walks the graph and assigns each op to a backend that can execute it.

- **Backends** are concrete implementations for one hardware target. Each registers a list of ops it supports.
- **Hardware** does the work.

The layering is what makes the project portable. Adding a new hardware target is a matter of implementing a backend; the llama runtime does not change at all.

What a tensor is, in ggml

A `ggml_tensor` is a struct with roughly this shape:

```
struct ggml_tensor {
    enum ggml_type    type;           // F32, F16, Q4_K, Q8_0, ...
    int64_t           ne[4];          // shape, outermost first
    size_t            nb[4];          // strides (bytes)
    enum ggml_op       op;            // how this tensor is computed
    struct ggml_tensor *src[GGML_MAX_SRC]; // inputs
    void              *data;          // raw bytes (mmap'd for weights)
    char              name[64];
    // ... plus fields for scheduling, grad, etc.
};
```

Two flavors exist: **leaves** (weights or inputs — their `op` is `GGML_OP_NONE` and `data` points to real memory) and **internal nodes** (their `op` is something like `GGML_OP_MUL_MAT`, and `data` is a view onto a compute buffer that the backend fills).

A tensor with `type = GGML_TYPE_Q4_K` does *not* carry F32 values in `data`. It carries the on-disk quantization layout — the same bytes the GGUF file had. The backend's `MUL_MAT` implementation knows how to multiply a quantized matrix by an F32 vector without ever materialising the full matrix.

The graph

A graph is just a list of tensors in dependency order, plus a `ggml_cgraph` header. You build it by calling op functions that take other tensors and return new ones:

```
// hidden = x @ W
struct ggml_tensor *hidden = ggml_mul_mat(ctx, W, x);
// hidden = RMSNorm(hidden)
hidden = ggml_rms_norm(ctx, hidden, eps);
```

```
// Append to graph
ggml_build_forward_expand(graph, hidden);
```

Nothing runs yet. The graph is a plan. To execute, hand it to a scheduler and compute buffer:

```
ggml_backend_sched_graph_compute(sched, graph);
```

At that point, the scheduler walks the graph and calls each backend's execute function for the ops it owns.

Backends

A backend exposes a small interface:

```
struct ggml_backend_i {
    const char * (*get_name)(...);
    bool (*supports_op)(ggml_backend_t, const ggml_tensor *);
    enum ggml_status (*graph_compute)(ggml_backend_t, ggml_cgraph *);
    // ... memory alloc, buffer mgmt, events, etc.
};
```

Implementations:

Backend

File

Hardware

CPU

```
ggml-cpu/ggml-cpu.c
```

x86, ARM, POWER

CUDA

```
ggml-cuda/ggml-cuda.cu
```

NVIDIA GPUs

Metal

```
ggml-metal/ggml-metal.m
```

Apple GPUs

Vulkan

```
ggml-vulkan/ggml-vulkan.cpp
```

AMD, Intel, Android

SYCL

```
ggml-sycl/ggml-sycl.cpp
```

Intel GPUs

HIP

```
ggml-cuda/*.cu (aliased)
```

AMD ROCm

CANN

```
ggml-cann/ggml-cann.cpp
```

Huawei NPUs

All seven implement the same interface. Fewer than half are built into any given binary — a release build picks one (or two, for mixed CPU+GPU) via the CMake flags from chapter 2.

The scheduler

When multiple backends are active, the scheduler partitions the graph. Its rules, simplified:

1. **If a backend can run an op and has the inputs nearby, prefer it.** Don't bounce tensors between devices without a reason.
2. **Keep small ops on CPU.** Launching a CUDA kernel for a single softmax is usually slower than running it on the host.
3. **Respect the layer-offload setting.** When you pass `-ngl 20`, the scheduler is told "place blocks 0..19 on GPU, 20..N on CPU."

Bounces between backends go through an explicit copy. You can see them in `GGML_SCHED_DEBUG=1` output; they are the "this backend → that backend" lines. Too many bounces is a classic cause of slow mixed- offload runs.

Op support

Not every backend supports every op. `supports_op` is how a backend says "I can't do this one; ask someone else." A missing op on the primary backend falls back to CPU, which for dense ops is a big slowdown. Running with `GGML_LOG_LEVEL=4` tells you which ops are falling back.

Warning

The first time you run a new architecture with a given backend, watch for "fallback to CPU" lines. A recent enough llama.cpp usually supports all of llama's ops on CUDA and Metal, but niche quantization types occasionally lag — especially on Vulkan and SYCL.

Memory and buffers

Weights live in a **backend buffer**. For CPU backends, that buffer is just host RAM (possibly mmap'd from the GGUF). For GPU backends, it's device memory (CUDA, Metal, ...). The loader decides at init time where each tensor goes and uploads accordingly.

The *compute buffer* — where intermediate activations live during a graph computation — is separate. It is usually small (a few hundred MB is typical for a 7B) and is allocated once per context.

Three things to take from this chapter

Tip

1. **A graph is a plan, not a computation.** The backend executes it.
2. **Quantized tensors stay quantized in memory.** Backends have kernels that multiply against them directly.
3. **Fallbacks are visible.** `GGML_LOG_LEVEL=4` is your friend when the GPU path looks slower than expected.

Exercises

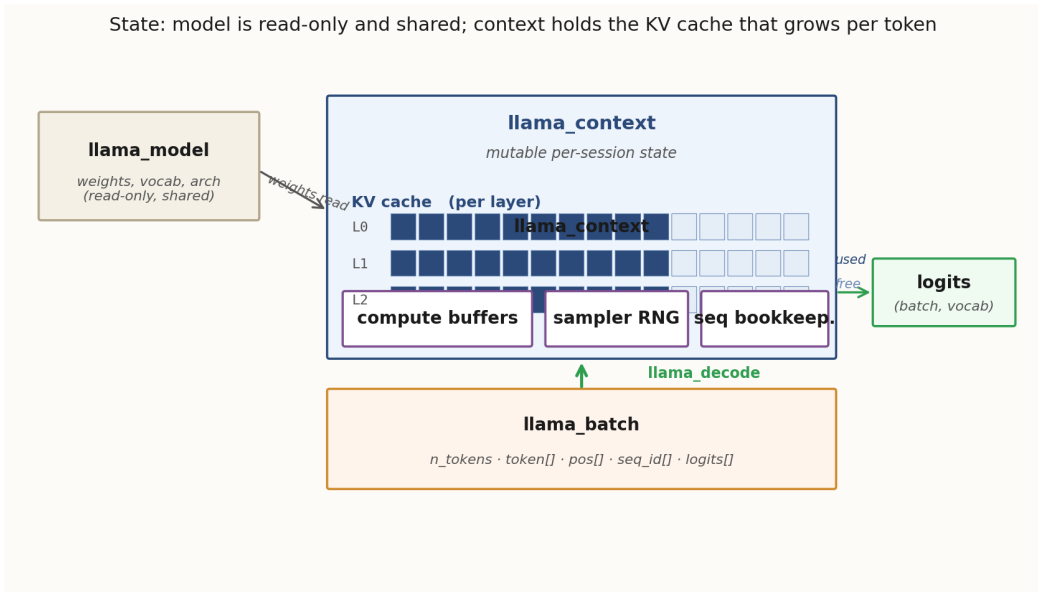
1. Build with both CPU and CUDA. Run `llama-bench -ngl 1 -ngl 5 -ngl 20` and read the output with `GGML_SCHED_DEBUG=1`. Count backend bounces at each offload level.
2. Pick one op (say `RMSNorm`). Find its implementation in `ggml-cpu`, `ggml-cuda`, and `ggml-metal`. What is the same across the three? What is different?
3. Write a 30-line C program that uses only `ggml.h` to build a graph `C = A @ B` for two small F32 matrices, runs it on the CPU backend, and prints the result. No llama, no model.

CHAPTER

Context, Batch, and the KV Cache

A GGUF file is read-only and shared. The state that changes between tokens — the growing sequence of K and V projections, the current batch, the sampler RNG — lives in a `llama_context`. Understanding the three moving pieces (*context*, *batch*, *KV cache*) is the difference between "my code segfaults on the second request" and "my code handles multi-turn chat correctly."

The three-object picture



A `llama_context` owns a KV cache sized by `n_ctx`; a `llama_batch` describes which tokens to evaluate in this call; `llama_decode` consumes the batch, advances the KV cache, and writes logits. Multiple contexts share the model but never share the cache.

Pin these three mental objects:

- `llama_model` — the weights. Immutable after load. Shared freely between threads.
- `llama_context` — the mutable state: KV cache, scratch buffers, sampler RNG seed, sequence bookkeeping. Not thread-safe; one context per conversation.
- `llama_batch` — a transient description of the tokens being evaluated in the current call. Points into user-owned arrays.

Every `llama_decode` consumes one batch, advances the context's KV cache, and produces logits. That is the entire contract.

Creating a context

```
llama_model_params mp = llama_model_default_params();
mp.n_gpu_layers = 99; // offload all layers
llama_model *model = llama_model_load_from_file("M.gguf", mp);

llama_context_params cp = llama_context_default_params();
```

```

cp.n_ctx    = 4096;           // max context length
cp.n_batch  = 512;           // prefill batch size
cp.n_ubatch  = 512;          // micro-batch for attention
cp.n_seq_max = 1;            // parallel sequences
llama_context *ctx = llama_init_from_model(model, cp);

```

`n_ctx` is the biggest position index the KV cache will ever hold. If you are writing a chat app, set this to the chat's expected maximum length; every extra slot costs real memory.

`n_batch` is the biggest chunk you'll ever pass to `llama_decode` at prefill; `n_ubatch` is the tile size inside the attention kernel. For most single-user chat, `n_batch = n_ubatch = 512` is a reasonable default.

The KV cache, sized

For a model with `L` layers, `H_kv` KV heads, per-head dim `d`, and `n_ctx` positions, at F16:

```

kv_bytes = 2 × L × n_ctx × H_kv × d × 2    // K and V, F16 = 2 bytes

```

For Qwen-2.5 7B ($L=28$, $H_{kv}=4$, $d=128$) with $n_{ctx}=8192$:

```

2 × 28 × 8192 × 4 × 128 × 2 ≈ 470 MiB

```

That is the number to keep in the back of your mind when you write `cp.n_ctx = 32768`. The KV cache grows linearly with `n_ctx`, so quadrupling the context quadruples the cache memory.

For long-context use cases, llama.cpp supports quantizing the cache itself:

```

cp.type_k = GGML_TYPE_Q8_0; // instead of F16
cp.type_v = GGML_TYPE_Q8_0;

```

Q8_0 KV cache halves memory with nearly zero quality loss. Q4 variants exist but are more aggressive; measure on your task.

Feeding a batch

A batch is:

```
struct llama_batch {
    int32_t n_tokens;
    llama_token *token;    // [n_tokens] token IDs
    llama_pos *pos;        // [n_tokens] position index
    int32_t *n_seq_id;     // [n_tokens] #sequences this token belongs to
    llama_seq_id **seq_id; // [n_tokens][n_seq_id[i]] sequence IDs
    int8_t *logits;        // [n_tokens] 1 = return logits for this token
};
```

The usual helper to build one:

```
llama_batch batch = llama_batch_init(n_ctx, /*embd=*/0, /*n_seq_max=*/1);
for (int i = 0; i < prompt_tokens.size(); ++i) {
    batch.token  [batch.n_tokens] = prompt_tokens[i];
    batch.pos    [batch.n_tokens] = i;
    batch.n_seq_id[batch.n_tokens] = 1;
    batch.seq_id [batch.n_tokens][0] = 0;
    batch.logits [batch.n_tokens] = (i == prompt_tokens.size() - 1);
    batch.n_tokens++;
}
llama_decode(ctx, batch);
```

The `logits` flag matters: by default the runtime does *not* return logits for every input token, because you usually only need the last one (to sample the next token from). Set it to 1 on just the last prompt token for prefill; set it on every token for workloads that want a per-position loss.

Multi-sequence contexts

Set `cp.n_seq_max = 8` and a single context can hold up to 8 independent conversations. You mark each token with a `seq_id`; the KV cache partitions by ID. `llama-server` uses this to serve many users out of one context without re-allocating.

To clear a specific sequence (end of chat, abort):

```
llama_kv_cache_seq_rm(ctx, seq_id, -1, -1); // remove all its slots
```

Continuous prompt reuse

If two requests share a prefix, you can keep the KV cache and just add new tokens:

```
// First request: evaluate the full prompt.
llama_decode(ctx, first_batch);

// Second request, same system prompt + different user turn:
// - Keep positions [0..sys_len) in the cache as-is.
// - Shift user tokens to start at position sys_len.
// - Decode only the new tokens.
llama_kv_cache_seq_rm(ctx, 0, sys_len, -1); // drop prior user turn
llama_decode(ctx, new_user_batch);
```

The server does this automatically (`--slot-save-path` on disk; in memory by default). For embedded use, it is a small amount of code and a large performance win.

Position indices matter

The `pos` field is *not* just "index in this call." It is the absolute position in the sequence. That is how the model's RoPE knows how far along the conversation it is. If you feed `pos = [0, 1, 2]` for the prompt and then try to sample with `pos = [0]` again, you will confuse the model.

Warning

RoPE scaling (`--rope-freq-base` , `--rope-scaling`) changes how position indices are interpreted. Mixing scaled and unscaled positions in the same cache produces nonsense. Pick one scaling and stick with it for the life of the context.

Three things to take from this chapter

Tip

1. **Model is read-only; context is mutable.** Share models between threads; never share contexts.
2. **KV cache size scales with** $n_ctx \times H_{kv} \times L$. Do the arithmetic before you ask for a huge context.
3. **Prompt reuse is free performance.** If you know the prefix is stable, don't re-decode it.

Exercises

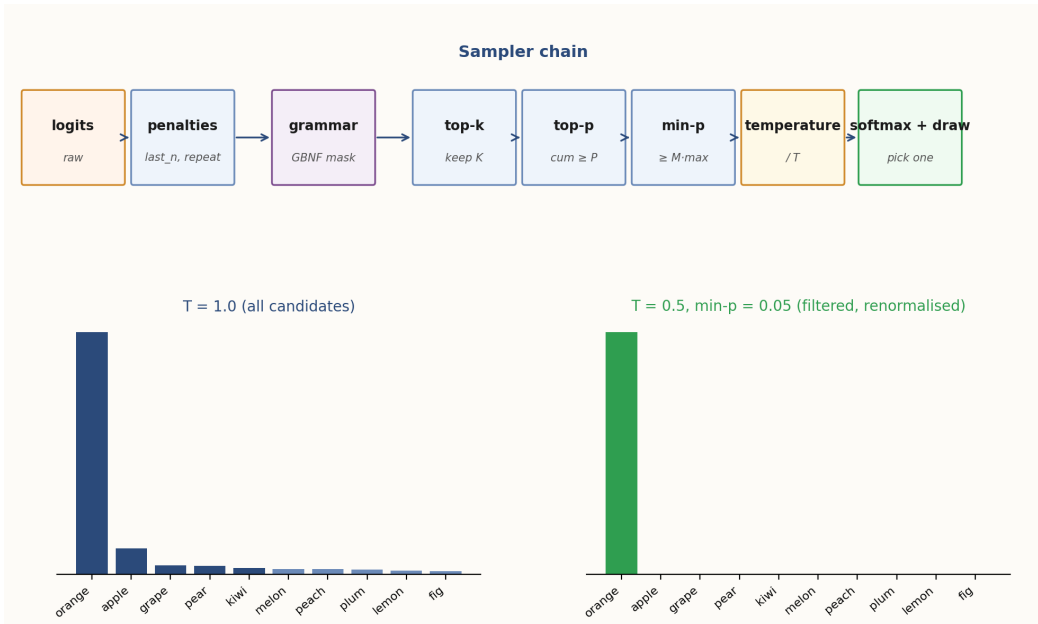
1. Measure the memory footprint of a single context as you vary `n_ctx` from 512 to 32768. Plot the curve. Does the slope match the formula in this chapter?
2. Build a toy chat loop that reuses the system prompt across turns. Time the second-turn prefill with and without reuse.
3. Read `llama-server`'s slot code (`tools/server/utils.hpp`). How does it decide which slot to drop when all are busy?

CHAPTER

The Sampler Chain

Each forward pass ends with a vector of logits — one score per token in the vocabulary. The sampler is what turns that vector into a single integer to append to the sequence. In `llama.cpp`, the sampler is not a single function: it is a chain of small, composable stages. This chapter shows the stages, when to use each, and how the chain maps to the public API.

The pipeline



Logits flow left-to-right: penalties adjust for recent tokens; grammar masks invalid continuations; truncation filters (top-k, top-p, min-p) thin the set; temperature scales; softmax produces probabilities; a draw picks one token.

A typical chain:

```
raw logits
  → apply logit_bias (user override)
  → repetition / frequency / presence penalties
  → grammar mask (zero-out invalid continuations)
  → top-k filter
  → top-p (nucleus) filter
  → min-p filter
  → temperature
  → softmax
  → draw
```

Most of these can be omitted. A minimal chain for chat is `penalties → min-p → temperature → draw`. A minimal chain for deterministic code generation is `greedy` (pick the argmax).

The parameters that matter

Flag

What it does

```
--temp T
```

Divide logits by `T`. Lower = more deterministic.

```
--top-k K
```

Keep the K highest-logit candidates. 40 is typical.

```
--top-p P
```

Keep a cumulative-probability core summing to $\geq P$. 0.9 typical.

```
--min-p M
```

Drop any candidate below `M × max_prob`. 0.05 is a good floor.

```
--repeat-penalty
```

Divide repeated tokens' logits by this (>1) in a window.

```
--grammar FILE
```

Apply a GBNF mask so only grammatically valid tokens pass.

```
--seed N
```

RNG seed; `-1` for random per-run.

Two sensible presets cover most use:

- **Chat / creative:** `--temp 0.7 --min-p 0.05 --repeat-penalty 1.1`
- **Code / deterministic:** `--temp 0` (implies greedy)

Stacking every filter at once (top-k 40, top-p 0.9, typical 0.95, min-p 0.05) is almost always a mistake. The most restrictive filter dominates; the others just run extra code.

Building a chain in C

The C API builds the chain explicitly:

```
#include "llama.h"

uint32_t seed = 42;
```

```

llama_sampler *chain = llama_sampler_chain_init(
    llama_sampler_chain_default_params());

llama_sampler_chain_add(chain,
    llama_sampler_init_penalties(
        /*last_n=*/64, /*repeat=*/1.1f,
        /*freq=*/0.0f, /*presence=*/0.0f));

llama_sampler_chain_add(chain, llama_sampler_init_top_k(40));
llama_sampler_chain_add(chain, llama_sampler_init_top_p(0.9f, /*min_keep=*/1));
llama_sampler_chain_add(chain, llama_sampler_init_min_p(0.05f, 1));
llama_sampler_chain_add(chain, llama_sampler_init_temp(0.7f));
llama_sampler_chain_add(chain, llama_sampler_init_dist(seed));

// Decode loop:
llama_token id = llama_sampler_sample(chain, ctx, /*idx=*/-1);
llama_sampler_accept(chain, id); // lets stateful stages observe

```

The final stage must always be a "distribution" stage (`llama_sampler_init_dist` for a standard stochastic draw, or `llama_sampler_init_greedy` for argmax). The earlier stages are optional filters.

Every stage is a small struct with function pointers; you can write your own by implementing `llama_sampler_i`. The chain treats user samplers and built-in ones identically.

Grammars (GBNF)

A grammar is a context-free description of legal output. At each step, the sampler walks the grammar to compute which token IDs can possibly come next and zeros out the logits of all others. The model is then forced to produce grammatically valid text.

A minimal JSON grammar (file `json.gbnf`):

```

root    ::= object
value   ::= object | array | string | number | "true" | "false" | "null"
object  ::= "{" ws (pair ("," ws pair)*)? "}"
pair    ::= string ws ":" ws value
array   ::= "[" ws (value ("," ws value)*)? "]"
string  ::= "\"" ([^"\\] | "\\" .)* "\""
number  ::= "-"? [0-9]+ ( "." [0-9]+ )?
ws      ::= [ \t\n]*

```

Use it:

```
./build/bin/llama-cli -m M.gguf \  
  --grammar-file json.gbnf \  
  -p "Return the capital of France: "
```

The server takes the same grammar, or a JSON Schema (which it translates to GBNF internally).

Warning

A tight grammar can hurt output quality: if the schema forbids a field the model wanted to produce, it will pick a low-probability valid alternative instead. Start loose and tighten.

Debugging a weird output

Two moves, in order:

1. **Drop temperature to 0.** If greedy output is sane, your sampler setting is the bug. If greedy output is also wrong, the issue is upstream (prompt, template, quantization).
2. **Read the top-k probabilities.** With the HTTP server, set `logprobs: true` on the request; the JSON reply shows what the sampler saw.

Three things to take from this chapter

Tip

1. **The sampler is a chain, not a function.** You mix stages to taste.
2. `min-p` is the new default. It adapts to model confidence and usually replaces top-p.
3. **Grammars are free correctness** when the output shape is strict. Use them for JSON, enums, SQL keywords.

Exercises

1. Write a GBNF that forces the model to emit exactly one line in the form `NOUN: VERB.` with all-caps words. Test it.
2. Implement top-p in ~20 lines of Python on a synthetic logit array. Verify against llama.cpp with the same seed.
3. Build a sampler chain that has penalties first, grammar second, temperature third. Swap the order and observe: what is order-sensitive and what is not?

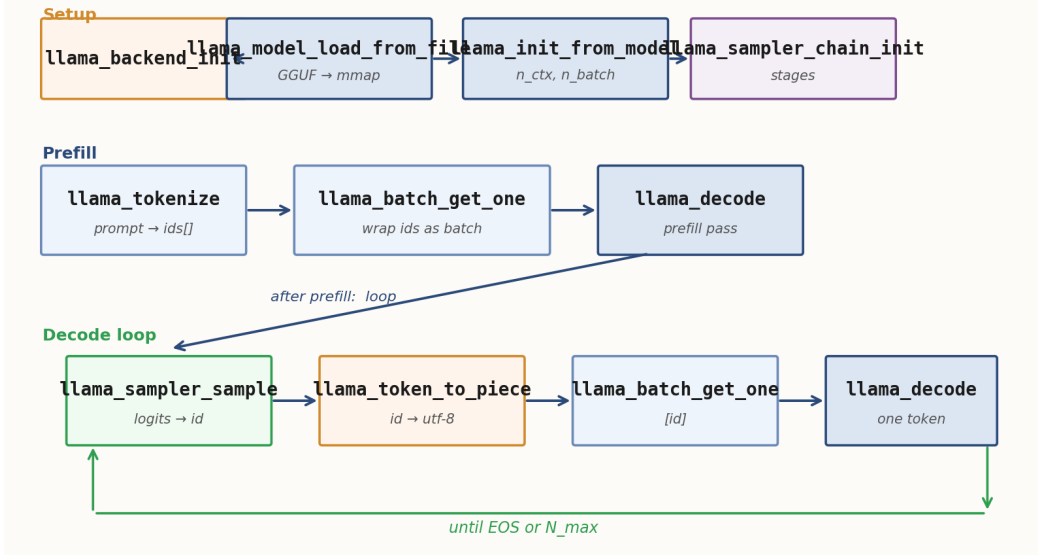
CHAPTER

Embedding llama.cpp in C++

The C API in `include/llama.h` is the project's actual public interface. Every binding in every other language ultimately calls the same functions. This chapter walks through a minimal but real C++ program that loads a GGUF, runs a prompt, and streams tokens — about 100 lines, end-to-end, with no CLI-layer magic.

What we are building

A minimal C++ decoder: setup once, prefill once, then loop sample/decode



A minimal C++ program: tokenize prompt; prefill decode; sample one token; append; decode again; loop until EOS. Same sequence as the CLI, just visible.

Pipeline:

1. Load model from GGUF.
2. Create a context with a specific `n_ctx`.
3. Tokenize the prompt.
4. Call `llama_decode` once with the prompt batch (prefill).
5. Sample a token, print it, append to a one-token batch.
6. Call `llama_decode` again (decode). Repeat until EOS or `N_max`.

CMake project

Link directly against the already-built library:

```
# CMakeLists.txt
cmake_minimum_required(VERSION 3.14)
project(mycli CXX)
set(CMAKE_CXX_STANDARD 17)
```



```

add_executable(mycli main.cpp)

# Point these at the llama.cpp build tree:
target_include_directories(mycli PRIVATE
    /path/to/llama.cpp/include
    /path/to/llama.cpp/ggml/include
    /path/to/llama.cpp/common)
target_link_directories(mycli PRIVATE /path/to/llama.cpp/build/bin)
target_link_libraries(mycli PRIVATE llama ggml)

```

When distributing, prefer `find_package(Llama CONFIG)` if the upstream install step was used; this is the "plugged in" variant.

The full program

```

#include "llama.h"

#include <cstdio>
#include <cstring>
#include <string>
#include <vector>

static std::vector<llama_token> tokenize(const llama_vocab *v,
                                         const std::string &text,
                                         bool add_special) {
    // Pass 1: size only.
    int n = -llama_tokenize(v, text.c_str(), text.size(),
                           nullptr, 0, add_special, /*parse_special=*/true);
    std::vector<llama_token> out(n);
    llama_tokenize(v, text.c_str(), text.size(),
                  out.data(), out.size(), add_special, true);
    return out;
}

int main(int argc, char **argv) {
    if (argc < 3) {
        fprintf(stderr, "usage: %s MODEL.gguf \"PROMPT\"\n", argv[0]);
        return 1;
    }
    const char *model_path = argv[1];
    std::string prompt      = argv[2];

    llama_backend_init();

```

```

// registers backends

```

```

llama_log_set(nullptr, nullptr); // silence startup spam

// --- 1. load model -----
llama_model_params mp = llama_model_default_params();
mp.n_gpu_layers = 99; // offload everything if available
llama_model *model = llama_model_load_from_file(model_path, mp);
if (!model) { fprintf(stderr, "load failed\n"); return 1; }
const llama_vocab *vocab = llama_model_get_vocab(model);

// --- 2. context -----
llama_context_params cp = llama_context_default_params();
cp.n_ctx = 2048;
cp.n_batch = 512;
llama_context *ctx = llama_init_from_model(model, cp);

// --- 3. tokenize -----
auto tokens = tokenize(vocab, prompt, /*add_special=*/true);

// --- 4. prefill -----
llama_batch batch = llama_batch_get_one(tokens.data(), tokens.size());
if (llama_decode(ctx, batch) != 0) { return 2; }

// --- 5. sampler chain -----
llama_sampler *smpl = llama_sampler_chain_init(
    llama_sampler_chain_default_params());
llama_sampler_chain_add(smpl, llama_sampler_init_min_p(0.05f, 1));
llama_sampler_chain_add(smpl, llama_sampler_init_temp(0.7f));
llama_sampler_chain_add(smpl, llama_sampler_init_dist(42));

// --- 6. decode loop -----
const int N_max = 128;
for (int i = 0; i < N_max; ++i) {
    llama_token id = llama_sampler_sample(smpl, ctx, -1);
    if (llama_vocab_is_eog(vocab, id)) break;

    char buf[256];
    int n = llama_token_to_piece(vocab, id, buf, sizeof(buf),
                                /*lstrip=*/0, /*special=*/false);
    fwrite(buf, 1, n, stdout); fflush(stdout);

    // One-token batch for next step.
    llama_batch nb = llama_batch_get_one(&id, 1);
    if (llama_decode(ctx, nb) != 0) break;
}
printf("\n");

```

```

    llama_sampler_free(smpl);
    llama_free(ctx);
    llama_model_free(model);
    llama_backend_free();
    return 0;
}

```

Build and run:

```

cmake -B build && cmake --build build
./build/mycli qwen2.5-1.5b-instruct-q4_k_m.gguf "Write a haiku about compilers."

```

Output streams token-by-token, exactly as `llama-cli` does, in about 100 lines of code you control.

What the helpers do

- `llama_backend_init` / `llama_backend_free` — register and unregister the compiled-in backends. Call once per process.
- `llama_batch_get_one` — a convenience constructor that builds a `llama_batch` pointing into user memory with `pos` auto-assigned starting at the current `kv_cache_get_used_cells` position. Handy for single-sequence chat.
- `llama_token_to_piece` — turns a token ID into its UTF-8 string fragment. Some tokens (BPE byte pairs) are not a full character on their own, so always stream the raw bytes and let the terminal handle composition.
- `llama_vocab_is_eog` — true for the model's end-of-generation sentinel (`</s>` , `<|im_end|>` , etc.). Models often have several.

Managing lifetimes

A `llama_context` pins a large amount of memory (KV cache, compute buffers). Free it as soon as the conversation ends:

```

// RAII wrapper pattern:
struct LlamaCtx {
    llama_context *ptr;
    ~LlamaCtx() { if (ptr) llama_free(ptr); }
}

```

```
};
```

Models are cheap to keep around and expensive to reload. Prefer one long-lived `llama_model` and many short-lived `llama_context`s.

Threading model

- Models are immutable after load — share them freely.
- Contexts are **not** thread-safe. One context per thread, or explicit external locking.
- The CPU backend uses its own thread pool (controlled by `llama_set_n_threads`). You do not need to spawn your own threads to parallelise a single decode.

Tip

For a web service, keep one `llama_model` shared between workers, and give each worker its own `llama_context`. Memory-mapped weights keep this cheap: the OS shares the weight pages between contexts via the page cache.

Three things to take from this chapter

Tip

1. **The full API surface fits in one header.** You can read `include/llama.h` in an afternoon.
2. **Six calls are the core path.** Load, init, tokenize, decode, sample, decode-again.
3. **Model is shareable; context is not.** That split drives every threading decision.

Exercises

1. Extend the program to accept a second argument `--stream=false` that prints the entire completion at the end instead of streaming. Measure if it changes timing.
2. Add a simple chat loop: print `>`, read a line from stdin, reuse the same context, exit on empty line. How many lines of glue does it take?

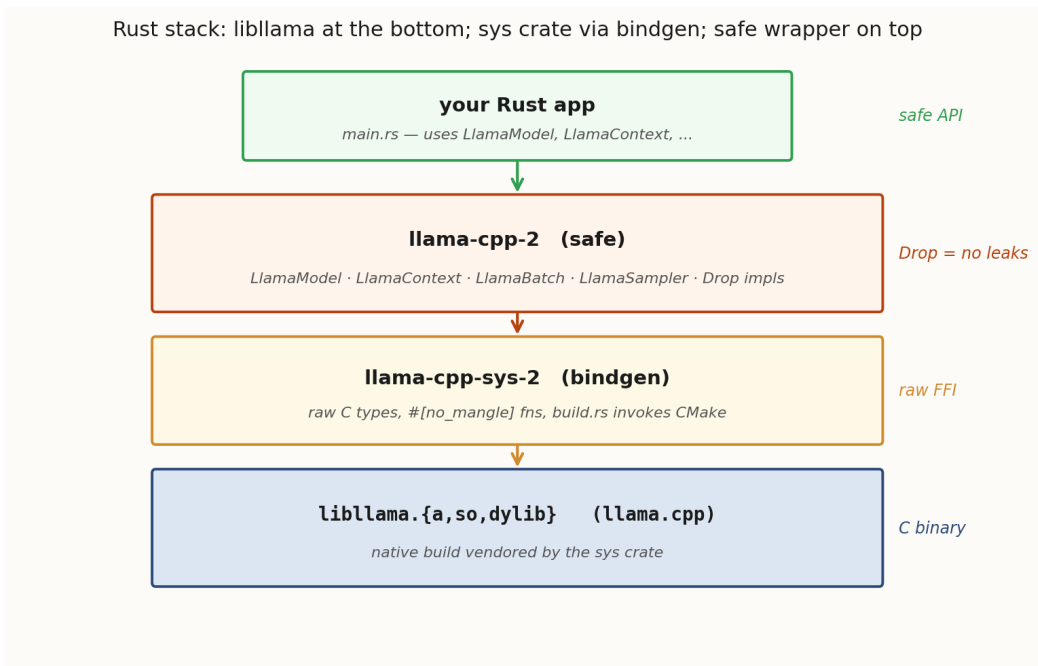
3. Replace `llama_batch_get_one` with a hand-constructed `llama_batch` and explicitly set `pos`, `seq_id`, and `logits`. Confirm the output is identical.

CHAPTER

Rust Bindings

From Rust, the best-maintained path into llama.cpp is the `llama-cpp-2` crate (with its `llama-cpp-sys-2` companion). It is a thin, safe wrapper over the C API. This chapter shows how to bring it into a Cargo project, how it maps to the C types you met in chapter 10, and how to write the same minimal generation loop idiomatically in Rust.

What the binding gives you



The Rust stack: `llama-cpp-sys-2` is the raw bindgen-generated FFI; `llama-cpp-2` is the safe wrapper with Rust types and `Drop` impls; your application sees `LlamaModel`, `LlamaContext`, `LlamaBatch`, `LlamaSampler`.

Two crates cooperate:

- `llama-cpp-sys-2` — generated by `bindgen` from `llama.h`; it exposes raw C types and functions, plus a build script that compiles llama.cpp from a vendored copy.
- `llama-cpp-2` — a safe layer with Rust-idiomatic types (`LlamaModel`, `LlamaContext`, `LlamaBatch`, `LlamaSampler`), lifetimes that enforce "model outlives context," and `Drop` implementations that free resources automatically.

Versions move in lockstep with upstream llama.cpp; always check the crate's README for which commit it vendors.

Cargo setup

```
# Cargo.toml
[package]
name = "mycli"
version = "0.1.0"
edition = "2021"

[dependencies]
llama-cpp-2 = "0.1"           # check crates.io for the latest
anyhow      = "1"
```

The sys crate builds llama.cpp from source on the first compile, which takes a few minutes. Cargo features choose the backend:

```
[dependencies.llama-cpp-2]
version = "0.1"
default-features = false
features = ["cuda"]           # or "metal", "vulkan", "sycl", ...
```

On macOS, `metal` is on by default. On Linux, CUDA requires `CUDA_PATH` in the environment. If compilation fails, the sys crate's `build.rs` prints the exact CMake command it attempted — copy-paste it to diagnose.

The full program

```
use anyhow::Result;
use llama_cpp_2::context::params::LlamaContextParams;
```

```

use llama_cpp_2::llama_backend::LlamaBackend;
use llama_cpp_2::llama_batch::LlamaBatch;
use llama_cpp_2::model::params::LlamaModelParams;
use llama_cpp_2::model::{AddBos, LlamaModel, Special};
use llama_cpp_2::sampling::LlamaSampler;
use std::num::NonZeroU32;
use std::path::PathBuf;
use std::io::{self, Write};

fn main() -> Result<()> {
    let args: Vec<String> = std::env::args().collect();
    let model_path = PathBuf::from(&args[1]);
    let prompt = args.get(2).cloned().unwrap_or_else(|| "Hello".into());

    // 1. Initialise the backend (registers CPU/CUDA/Metal as built).
    let backend = LlamaBackend::init()?;

    // 2. Load the model.
    let model_params = LlamaModelParams::default()
        .with_n_gpu_layers(99);
    let model = LlamaModel::load_from_file(&backend, model_path, &model_params)?;

    // 3. Create a context.
    let ctx_params = LlamaContextParams::default()
        .with_n_ctx(Some(NonZeroU32::new(2048).unwrap()))
        .with_n_batch(512);
    let mut ctx = model.new_context(&backend, ctx_params)?;

    // 4. Tokenize the prompt.
    let tokens = model.str_to_token(&prompt, AddBos::Always)?;

    // 5. Prefill.
    let mut batch = LlamaBatch::new(512, 1);
    let last_index = tokens.len() as i32 - 1;
    for (i, tok) in tokens.into_iter().enumerate() {
        let last = i as i32 == last_index;
        batch.add(tok, i as i32, &[0], last)?;
    }
    ctx.decode(&mut batch)?;

    // 6. Sampler chain.
    let mut sampler = LlamaSampler::chain_simple([
        LlamaSampler::min_p(0.05, 1),
        LlamaSampler::temp(0.7),
        LlamaSampler::dist(42),
    ])

```

```

]);

// 7. Decode loop.
let mut n_cur = batch.n_tokens();
let n_max = n_cur + 128;
let stdout = io::stdout();
let mut out = stdout.lock();

while n_cur < n_max {
    let token = sampler.sample(&ctx, batch.n_tokens() - 1);
    sampler.accept(token);
    if model.is_eog_token(token) { break; }

    let piece = model.token_to_str(token, Special::Tokenize)?;
    out.write_all(piece.as_bytes())?;
    out.flush()?;

    batch.clear();
    batch.add(token, n_cur, &[0], true)?;
    ctx.decode(&mut batch)?;
    n_cur += 1;
}
println!();
Ok(())
}

```

`cargo run --release -- model.gguf "Write a haiku about compilers."` and you will see the same token-by-token stream as the C++ version.

Where Rust helps

- **Drop order.** `LlamaContext` holds a reference to `LlamaModel`. The borrow checker enforces that the model outlives every context, so you cannot accidentally free the weights while a conversation is still open.
- **No raw pointers.** Tokens are `LlamaToken` (a typed wrapper), not `i32`. Batches own their buffers.
- **Error handling.** Every fallible C call becomes a `Result` with a descriptive error, not a -1 return.

Where it hurts

- **Versions drift fast.** The crate follows upstream but lags by days to weeks. Pin exact versions in `Cargo.toml` for anything you plan to ship.
- **Feature matrix is real.** Not every upstream build flag is exposed as a Cargo feature yet. When you need `GGML_BLAS=ON` or a new backend, you sometimes need to drop to `llama-cpp-sys-2` and set CMake defines in a `build.rs` override.
- **Compile times.** The `sys` crate builds `llama.cpp` itself; expect a few minutes on the first compile of a workspace.

Async and server-shaped use

The crate is synchronous by design; wrap calls in `tokio::task::spawn_blocking` for web servers. One long-lived `LlamaModel` + many short-lived `LlamaContext` s is the pattern, same as in C++. A common helper is a channel of "request → reply" messages sent to a dedicated worker thread that owns the contexts.

Note

For batch serving (many concurrent users on one context), look at the `llama-server` HTTP frontend rather than re-implementing its slot allocator in Rust. You can still use Rust on the *client* side and let the server handle concurrency.

Alternative crates

- `llm` — a higher-level, more opinionated wrapper, but historically lagged upstream by further. Good for quick scripts.
- `mistralrs` — a parallel Rust inference engine; shares some ideas with `llama.cpp` but is a separate project.
- `candle` — Hugging Face's pure-Rust tensor stack; not based on `llama.cpp`. Mentioned for completeness.

If your constraint is "ride `llama.cpp` tightly," `llama-cpp-2` is the right choice. Otherwise consider whether you need `llama.cpp` at all.

Three things to take from this chapter

Tip

1. `llama-cpp-2` is the **close-to-C wrapper**. Safe, idiomatic, and never more than one layer of indirection from `llama.h`.
2. **Compile times front-load**. Plan for a one-off slow build; subsequent rebuilds are fast.
3. **Mirror the C ownership**. One long-lived model, many short-lived contexts — the borrow checker will keep you honest.

Exercises

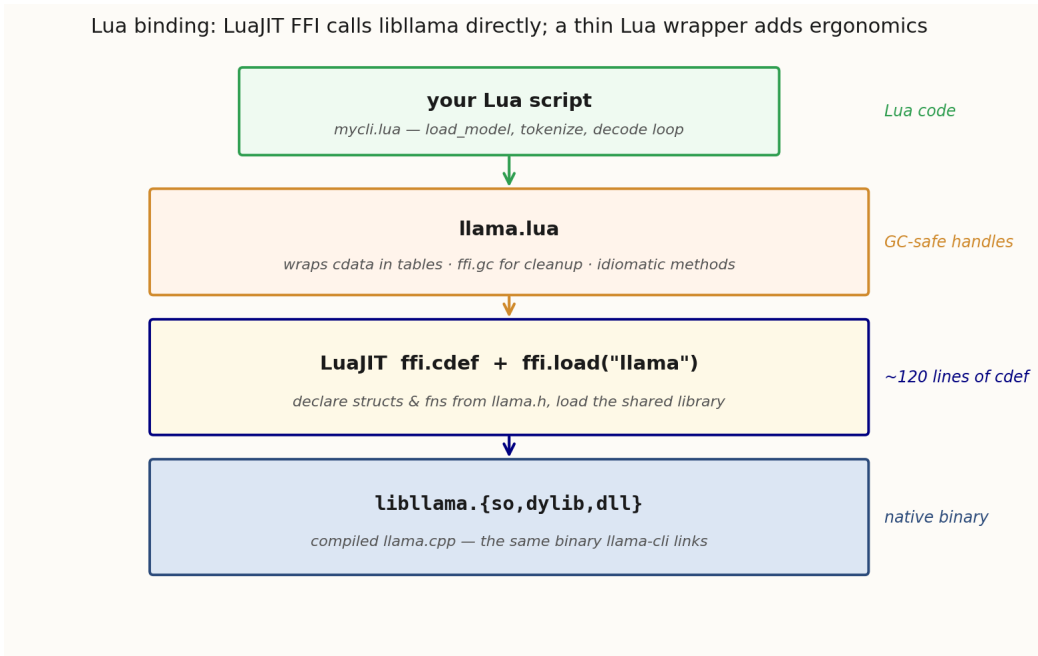
1. Port your C++ chat loop from chapter 10 to Rust. Which lines had the biggest drop in code volume? Which had the biggest rise?
2. Add a `#[tokio::main]` HTTP endpoint using `axum` that streams tokens to the client via SSE. Use `spawn_blocking` for the decode call.
3. Read `llama-cpp-2`'s `lib.rs`. How are `LlamaBatch` and `LlamaSampler` kept from outliving the context they were created against?

CHAPTER

Lua Bindings

Lua is the odd one out: there is no single blessed binding, but LuaJIT's FFI lets you call the C API directly with very little glue. In about 120 lines of Lua you can load a GGUF and stream tokens. This chapter shows how, with enough structure that you can grow the binding as your needs grow.

The shape of an FFI binding



LuaJIT FFI calls straight into libllama.so/.dylib using cdef-declared function signatures; a thin Lua module wraps the opaque pointers in tables with metatables for `__gc`-based cleanup; user code sees idiomatic Lua methods.

Three layers:

- `ffi.cdef[[ ... ]]` — declare the C structs and functions you plan to call.
- `ffi.load("llama")` — load `libllama.so` / `libllama.dylib` / `llama.dll`.
- **A small Lua module** — wraps raw pointers in tables, gives them methods, and uses `ffi.gc(ptr, dtor)` so Lua's GC frees them.

Vanilla Lua (5.1/5.3/5.4) can also do this through `luaaffib` or a hand-rolled C module, but LuaJIT is the pragmatic choice — it is several times faster and ships with ffi out of the box.

Declaring the C API

You do not need every function. The minimum to generate tokens:

```

local ffi = require("ffi")

ffi.cdef[[
    // Opaque handles.
    typedef struct llama_model      llama_model;
    typedef struct llama_context    llama_context;
    typedef struct llama_vocab      llama_vocab;
    typedef struct llama_sampler    llama_sampler;
    typedef int32_t                 llama_token;
    typedef int32_t                 llama_pos;
    typedef int32_t                 llama_seq_id;

    // Params structs – treat as opaque; we pass defaults through.
    typedef struct llama_model_params { char _pad[256]; } llama_model_params;
    typedef struct llama_context_params { char _pad[512]; } llama_context_params;
    typedef struct llama_sampler_chain_params {
        bool no_perf;
    } llama_sampler_chain_params;

    typedef struct llama_batch {
        int32_t      n_tokens;
        llama_token  *token;
        float        *embd;
        llama_pos    *pos;
        int32_t      *n_seq_id;
        llama_seq_id **seq_id;
        int8_t       *logits;
    } llama_batch;
]]

```

Then the functions, split into lifecycle, tokens, decode, and sampler groups:

```

ffi.cdef[[
    // Lifecycle.
    void      llama_backend_init(void);
    void      llama_backend_free(void);
    llama_model_params llama_model_default_params(void);
    llama_model * llama_model_load_from_file(const char *path,
                                              llama_model_params params);

    void      llama_model_free(llama_model *);
    const llama_vocab * llama_model_get_vocab(const llama_model *);
    llama_context_params llama_context_default_params(void);
    llama_context * llama_init_from_model(llama_model *,
                                          llama_context_params);
]]

```

```

    void                llama_free(llama_context *);
}]

```

```

ffi.cdef[[
    // Tokens.
    int32_t llama_tokenize(const llama_vocab *, const char *text,
                           int32_t text_len,
                           llama_token *tokens, int32_t n_tokens_max,
                           bool add_special, bool parse_special);
    int32_t llama_token_to_piece(const llama_vocab *, llama_token,
                                 char *buf, int32_t length,
                                 int32_t lstrip, bool special);
    bool    llama_vocab_is_eog(const llama_vocab *, llama_token);
]]

```

```

ffi.cdef[[
    // Decode + sampler.
    llama_batch llama_batch_get_one(llama_token *, int32_t n_tokens);
    int32_t      llama_decode(llama_context *, llama_batch);

    llama_sampler_chain_params llama_sampler_chain_default_params(void);
    llama_sampler * llama_sampler_chain_init(llama_sampler_chain_params);
    void           llama_sampler_chain_add(llama_sampler *, llama_sampler *);
    void           llama_sampler_free(llama_sampler *);
    llama_sampler * llama_sampler_init_min_p(float, size_t);
    llama_sampler * llama_sampler_init_temp(float);
    llama_sampler * llama_sampler_init_dist(uint32_t);
    llama_token     llama_sampler_sample(llama_sampler *, llama_context *, int32_t);
    void            llama_sampler_accept(llama_sampler *, llama_token);
]]

local L = ffi.load("llama")

```

The `_padding` trick in the params structs is a compromise: the real field layout shifts between llama.cpp versions, and declaring only the size keeps the binding working against new releases as long as you pass defaults through the C helpers. If you need to tweak a field, declare its full struct for the version you target.

A minimal wrapper module

```

-- llama.lua
local ffi = require("ffi")
local C = require("llama_cdefs") -- the cdef + ffi.load above

local M = {}

function M.init()
    C.llama_backend_init()
end

function M.load_model(path)
    local params = C.llama_model_default_params()
    local m = C.llama_model_load_from_file(path, params)
    assert(m ~= nil, "model load failed")
    ffi.gc(m, C.llama_model_free)
    return m
end

function M.new_context(model)
    local p = C.llama_context_default_params()
    local c = C.llama_init_from_model(model, p)
    ffi.gc(c, C.llama_free)
    return c
end

function M.tokenize(vocab, text)
    local buf = ffi.new("int32_t[?]", #text + 8)
    local n = C.llama_tokenize(vocab, text, #text, buf, #text + 8,
                              true, true)
    assert(n >= 0, "tokenize failed")
    return buf, n
end

function M.piece(vocab, token)
    local buf = ffi.new("char[256]")
    local n = C.llama_token_to_piece(vocab, token, buf, 256, 0, false)
    return ffi.string(buf, n)
end

return M

```

`ffi.gc` is the important call. It attaches a destructor to the cdata pointer so that when Lua's GC reclaims the Lua table wrapping it, the C resource is freed too. Without it, you would leak memory every time a script exits.

The full program

```
-- mycli.lua
local ffi = require("ffi")
local C = require("llama_cdefs")
local M = require("llama")

local model_path = arg[1]
local prompt = arg[2] or "Write a haiku about compilers."

M.init()
local model = M.load_model(model_path)
local vocab = C.llama_model_get_vocab(model)
local ctx = M.new_context(model)

-- Tokenize prompt and prefill.
local tokens, n = M.tokenize(vocab, prompt)
local batch = C.llama_batch_get_one(tokens, n)
assert(C.llama_decode(ctx, batch) == 0)

-- Sampler chain.
local params = C.llama_sampler_chain_default_params()
local smpl = C.llama_sampler_chain_init(params)
ffi.gc(smpl, C.llama_sampler_free)
C.llama_sampler_chain_add(smpl, C.llama_sampler_init_min_p(0.05, 1))
C.llama_sampler_chain_add(smpl, C.llama_sampler_init_temp(0.7))
C.llama_sampler_chain_add(smpl, C.llama_sampler_init_dist(42))

-- Decode loop.
local one = ffi.new("int32_t[1]")
local N_max = 128
for i = 1, N_max do
    local id = C.llama_sampler_sample(smpl, ctx, -1)
    if C.llama_vocab_is_eog(vocab, id) then break end
    io.write(M.piece(vocab, id))
    io.flush()
    one[0] = id
    local nb = C.llama_batch_get_one(one, 1)
    if C.llama_decode(ctx, nb) ~= 0 then break end
end
io.write("\n")
```

Run it:

```
LD_LIBRARY_PATH=/path/to/llama.cpp/build/bin luajit mycli.lua model.gguf
```

On macOS, set `DYLD_LIBRARY_PATH` instead.

Why LuaJIT

The vanilla Lua C API is fine but verbose: every exported function becomes a `static int l_name(lua_State*)`, you push/pop values, handle errors. With LuaJIT's FFI the C header is almost directly usable, and calls are nearly as fast as native C — LuaJIT's tracing compiler can inline them.

For vanilla Lua (5.1 / 5.3 / 5.4), options include:

- `luaaffib` — Facebook's FFI port for stock Lua. Similar API but slower than LuaJIT's.
- **Hand-rolled C module** — a `llama.c` with `luaL_Reg` entries. More code, fewer runtime dependencies.
- **Community bindings** — Search for `llama-cpp-lua` on GitHub; projects exist, but mileage varies.

Gotchas

- **String lifetimes.** `ffi.string` copies the bytes; you can let the `char[]` buffer go immediately after.
- **Struct layout drift.** Padding sizes change between versions. Pin a commit and regenerate your cdefs when you bump.
- **Thread safety.** LuaJIT's ffi can be called from multiple Lua states, but contexts remain single-threaded: do not share a `llama_context` across coroutines without scheduling.

Warning

If `ffi.load("llama")` returns "library not found," LuaJIT is looking in the system paths. Either set `LD_LIBRARY_PATH` (Linux) / `DYLD_LIBRARY_PATH` (macOS), or pass the absolute path to `ffi.load` explicitly.

Three things to take from this chapter

Tip

1. **LuaJIT + FFI is the pragmatic path.** Minimal glue, native performance.
2. `ffi.gc` **is required for every handle.** Attach a destructor on the same line you create the pointer.
3. **Pin a llama.cpp commit.** Padding sizes change with releases; regenerate cdefs when you bump.

Exercises

1. Add `llama_set_n_threads` to the cdef and a helper in the module. Plot tokens/sec as you vary thread count from 1 to your core count.
2. Wrap the program in a small REPL: `>` prompt, print tokens, reset on `!reset`, exit on `!quit`. How do you keep the KV cache across turns?
3. Measure the overhead of LuaJIT FFI vs. the pure C program from chapter 10 on the same model. Where does the gap land — in decode, in dispatch, or somewhere else?

CHAPTER

Appendix A: C API Cheat-Sheet

The functions that appear most often in embedded use, grouped by lifecycle. Signatures come from `include/llama.h`; they change occasionally — when in doubt, the header is the source of truth.

Startup / teardown

```
void llama_backend_init(void);
```

```
void llama_backend_free(void);
```

Call once per process. They register and unregister the built-in backends.

Model

```
llama_model_params llama_model_default_params(void);
llama_model * llama_model_load_from_file(const char *path,
                                          llama_model_params params);
void llama_model_free(llama_model *model);
const llama_vocab * llama_model_get_vocab(const llama_model *model);

int32_t llama_model_n_ctx_train(const llama_model *);
int32_t llama_model_n_embd(const llama_model *);
int32_t llama_model_n_layer(const llama_model *);
int32_t llama_model_n_head(const llama_model *);
int32_t llama_model_n_head_kv(const llama_model *);
```

`llama_model_params` fields you'll touch:

- `int32_t n_gpu_layers` — how many transformer layers to place on the GPU.
99 = all.
- `bool use_mmap` — `true` by default; `false` forces a full read.
- `bool use_mlock` — pins weights in RAM.

Context

```
llama_context_params llama_context_default_params(void);
llama_context * llama_init_from_model(llama_model *, llama_context_params);
void llama_free(llama_context *);

void llama_set_n_threads(llama_context *, int32_t n_threads, int32_t n_threads_batch);
uint32_t llama_n_ctx(const llama_context *);
```

`llama_context_params` fields you'll touch:

- `uint32_t n_ctx` — max position. 0 means "model's training ctx".
- `uint32_t n_batch` — max tokens per `llama_decode` call.

- `uint32_t n_ubatch` — tile size inside the attention kernel.
- `enum ggml_type type_k`, `type_v` — quantize the KV cache (`GGML_TYPE_F16`, `GGML_TYPE_Q8_0`, ...).

Tokens

```
int32_t llama_tokenize(const llama_vocab *, const char *text, int32_t text_len,
                      llama_token *out, int32_t out_max,
                      bool add_special, bool parse_special);

int32_t llama_token_to_piece(const llama_vocab *, llama_token,
                             char *buf, int32_t buf_len,
                             int32_t lstrip, bool special);

bool llama_vocab_is_eog(const llama_vocab *, llama_token);
llama_token llama_vocab_bos(const llama_vocab *);
llama_token llama_vocab_eos(const llama_vocab *);
```

`llama_tokenize` returns a negative number on overflow: pass it back in absolute value as `out_max`.

Batch

```
llama_batch llama_batch_init(int32_t n_tokens_alloc, int32_t embd,
                             int32_t n_seq_max);
llama_batch llama_batch_get_one(llama_token *tokens, int32_t n_tokens);
void llama_batch_free(llama_batch batch);

int32_t llama_decode(llama_context *, llama_batch batch);
float * llama_get_logits_ith(llama_context *, int32_t i);
```

`llama_batch_get_one` is the convenience path: positions start where the KV cache currently ends, with a single sequence. For multi-sequence or embedding workloads, build the batch manually with `llama_batch_init`.

KV cache

```

void llama_kv_cache_clear (llama_context *);
bool llama_kv_cache_seq_rm (llama_context *, llama_seq_id, llama_pos p0, llama_pos p1);
void llama_kv_cache_seq_cp (llama_context *, llama_seq_id src, llama_seq_id dst,
                             llama_pos p0, llama_pos p1);
void llama_kv_cache_seq_keep(llama_context *, llama_seq_id);
void llama_kv_cache_seq_shift(llama_context *, llama_seq_id,
                              llama_pos p0, llama_pos p1, llama_pos shift);
int32_t llama_kv_cache_get_used_cells(const llama_context *);

```

`p0 = -1` means "from start," `p1 = -1` means "to end." These are what `llama-server` uses to share a context between users.

Sampler chain

```

llama_sampler_chain_params llama_sampler_chain_default_params(void);
llama_sampler * llama_sampler_chain_init(llama_sampler_chain_params);
void llama_sampler_chain_add (llama_sampler *chain, llama_sampler *stage);
void llama_sampler_free (llama_sampler *);

llama_sampler * llama_sampler_init_greedy (void);
llama_sampler * llama_sampler_init_dist (uint32_t seed);
llama_sampler * llama_sampler_init_temp (float t);
llama_sampler * llama_sampler_init_top_k (int32_t k);
llama_sampler * llama_sampler_init_top_p (float p, size_t min_keep);
llama_sampler * llama_sampler_init_min_p (float p, size_t min_keep);
llama_sampler * llama_sampler_init_typical (float p, size_t min_keep);
llama_sampler * llama_sampler_init_penalties (int32_t last_n, float repeat,
                                              float freq, float presence);
llama_sampler * llama_sampler_init_grammar (const llama_vocab *,
                                           const char *grammar_str,
                                           const char *root_rule);

llama_token llama_sampler_sample(llama_sampler *, llama_context *, int32_t idx);
void llama_sampler_accept(llama_sampler *, llama_token);

```

The chain owns the stages added to it; freeing the chain frees everything in it.

Freeing things

Pair every `*_init` / `*_load` with its `*_free` / `*_free_XXX` :

- `llama_model_load_from_file` → `llama_model_free`
- `llama_init_from_model` → `llama_free`
- `llama_batch_init` → `llama_batch_free`
- `llama_sampler_chain_init` → `llama_sampler_free`
- `llama_backend_init` → `llama_backend_free`

`llama_batch_get_one` does **not** allocate — its arrays point into user memory. Do not pass its return to `llama_batch_free`.

CHAPTER

Appendix B: Glossary

A quick reference for terms used without ceremony in the text.

Attention — the operator that lets every token look at every earlier token. In llama.cpp, attention is computed per layer with `Q = xW_q`, `K = xW_k`, `V = xW_v`, then `softmax(QK^T/√d) · V`.

Backend — a GGML target (`cpu`, `cuda`, `metal`, `vulkan`, `sycl`, `hip`, `cann`). Picked at build time, loaded at runtime.

Batch (`llama_batch`) — the set of tokens submitted together to `llama_decode`. At prefill this is the whole prompt; at decode, a single token.

BOS / EOS / EOG — "beginning of sequence" / "end of sequence" / "end of generation" special tokens. Models often have multiple EOG tokens; use `llama_vocab_is_eog` to check any given ID.

BPE (Byte-Pair Encoding) — the tokenizer family used by most modern LLMs. Tokens are sub-word pieces; a single character can be split across multiple IDs.

BPW (bits per weight) — a file's average quantization density. `Q4_K_M` is ~4.9 BPW.

Chat template — a Jinja2 string stored in the GGUF metadata that formats `{system, user, assistant}` messages into the exact string the model was trained on.

Context (`llama_context`) — the runtime state object. Owns the KV cache, compute buffers, sampler RNG, and sequence bookkeeping.

Continuous batching — the server-side trick of running many users' tokens through a single `llama_decode` call, mixing prefill and decode freely. Requires `n_seq_max > 1`.

Decode — the per-token phase after prefill. One new token per `llama_decode` call; each one advances the KV cache by one position.

Embedding — a vector representation of text, extracted from the hidden state rather than from the output head. `llama-embedding` and the server's `/v1/embeddings` produce them.

F16 / BF16 / F32 — half / bfloat16 / single-precision floats. F16 is the default "full precision" in the GGUF ecosystem; BF16 arrives from many training stacks and is supported too.

FFN — feed-forward network; the second half of a transformer block. In modern models it is a gated SwiGLU: `(silu(xw_g) ■ xw_u) w_d`.

GBNF — GGML's flavor of Backus–Naur Form, used to constrain sampler output to a grammar.

GGML — the tensor library at the bottom of llama.cpp. Defines the graph, ops, and backends.

GGUF — the current file format for a llama.cpp model. One blob containing weights, tokenizer, and metadata.

GQA (Grouped-Query Attention) — fewer K/V heads than Q heads. Shrinks the KV cache proportionally.

i-quants / IQ — importance-matrix-aware quantization types (`IQ2_XS`, `IQ3_XXS`, `IQ4_XS`, ...). Best at sub-4 bits.

iMatrix — an "importance matrix" computed on a calibration corpus, used by `llama-quantize` to pick better quant codebooks.

K-quants — a family of quantization types (`Q2_K` – `Q6_K`) with super-block scales. The consumer default is `Q4_K_M`.

KV cache — cached K and V projections for every past token in a sequence, per layer. Lets decode be O(1) per token instead of O(n).

Layer / block — one transformer layer: RMSNorm → attention → RMSNorm → FFN, with residual adds. A 7B model has ~32 layers.

Logits — raw per-vocabulary scores produced by the final output projection, before softmax.

Memory mapping (`mmap`) — the OS trick of backing a virtual memory region with a file on disk. llama.cpp uses it so multiple processes can share one copy of the weights.

min-p — a sampling filter that drops candidates below `min_p × max_prob` . Usually the best single filter.

n_ctx — maximum number of positions the KV cache will hold. Decided at context creation; shrinks available memory linearly.

Offload (`-ngl N`) — placing the first N transformer layers on the GPU backend. 0 = pure CPU, 99 = everything on the GPU.

Perplexity — exponential of average negative log-likelihood on a text. `llama-perplexity` is the reference tool.

Prefill — the first `llama_decode` call that evaluates the entire prompt in a single batch. Compute-bound, dominated by matrix-multiply throughput.

Quantization — replacing F16/F32 weights with integers plus per-group scales. The reason 7B models fit in 4 GB of RAM.

RMSNorm — Root-Mean-Square normalization: $x / \text{rms}(x) \times \gamma$. Cheaper and sometimes better than LayerNorm; ubiquitous in modern LLMs.

RoPE (Rotary Position Embedding) — positional information encoded by rotating Q and K vectors by a frequency that depends on absolute position.

Sampler chain — the ordered set of stages that turn logits into a token: penalties, filters, temperature, draw.

SwiGLU — `silu(gate) ■ up` ; the three-matrix gated FFN used by llama, qwen, gemma, mistral, and most contemporaries.

Tensor (`ggml_tensor`) — a named, typed, n-D array inside GGML. Can be a weight (leaf) or a computation (internal node).

Token — an integer vocabulary index. Both input and output of the model are sequences of tokens.

Tokenizer — the component that turns text into tokens and back. In llama.cpp, the tokenizer is stored inside the GGUF.

Top-k / top-p — candidate-set filters: keep the K best, or the smallest set summing to P cumulative probability.

Unified memory — Apple silicon's shared CPU/GPU memory, where `mmap`'d weights are simultaneously visible to both sides without a copy.

Vocabulary — the ordered list of tokens for a model. Usually 32k–256k entries. Stored in GGUF metadata.